3. セキュリティアーキテクチャとエンジニアリング

3.1 セキュアな設計原則を使用して、エンジニアリングプロセスを調査、実装、管理する

No.	用語	概要・定義	具体例	CISSP試験での重要ポイント
1	**脅威モデリング（Threat Modeling）**	システムに対する潜在的な脅威、攻撃経路、脆弱性を体系的に特定・分析し、対策を検討するプロセス。	Webアプリケーションの設計段階でSTRIDEを用いて、なりすましや情報漏洩の脅威を洗い出す。	**設計段階から脅威を特定することが重要。** STRIDE、PASTA、攻撃ツリーなどの手法を理解する。
2	**最小権限（Least Privilege）**	ユーザーやプロセスに、業務遂行に必要な最小限の権限のみを付与する原則。	一般社員には必要な閲覧権限のみを付与し、管理者権限は必要な担当者に限定する。	権限を最小化することで、内部不正やアカウント侵害時の被害を抑制する。**Need to Knowとの違い**も理解する。
3	**多層防御（Defense in Depth：DiD）**	複数のセキュリティ対策を組み合わせ、一つの対策が突破されても他の対策で保護する設計思想。	ファイアウォール、IDS/IPS、EDR、アクセス制御、暗号化を組み合わせる。	**単一の防御策に依存しない。** 予防・検知・対応など、異なる役割の対策を組み合わせる。
4	**セキュアデフォルト（Secure Defaults）**	初期状態から安全な設定を適用し、明示的に許可されていない操作やアクセスを原則禁止する設計思想。	新規アカウントには管理者権限を与えず、不要なポートやサービスを初期状態で無効化する。	**デフォルト拒否（Default Deny）** が重要。初期設定のままでも安全性が確保される設計を目指す。
5	**フェイルセキュア（Fail Secure）**	システムに障害が発生した場合でも、セキュリティが維持される状態に移行する設計原則。	認証サーバーが停止した場合、新規の認証要求を許可せずアクセスを拒否する。	**障害発生時も機密性・完全性を維持する。** Fail Safeとの違いに注意。人命や物理的安全を優先する場面では設計判断が異なる。
6	**職務の分離（Separation of Duties：SoD）**	重要な業務や処理を複数の担当者に分割し、一人で不正を完結できないようにする内部統制の原則。	システム変更の申請者と承認者を別人にする。	**不正・誤操作の防止が目的。** Dual Control（二人による共同統制）やJob Rotation（職務ローテーション）との違いを理解する。
7	**シンプルに保つ（Keep It Simple）**	システムやセキュリティ機構を必要以上に複雑にせず、理解・検証・管理しやすい設計を維持する原則。	複雑なアクセス制御ルールを整理し、不要な例外設定を削減する。	**複雑性は設定ミスや脆弱性の原因になる。** Economy of Mechanism（機構の単純化）と関連する。

No.	用語	概要・定義	具体例	CISSP試験での重要ポイント
8	**信ぜよ、されど確認せよ（Trust but Verify）**	一定の信頼を前提としながらも、検証や監査によって信頼性を確認する考え方。	社内ユーザーのアクセスを許可しつつ、操作ログを監査し、不審な行動を検知する。	信頼していても検証を省略しない。**ゼロトラストとの思想的な違い**を理解する。
9	**ゼロトラスト（Zero Trust）**	ネットワークの内外という位置だけで暗黙の信頼を与えず、アクセス要求ごとに適切な認証・認可を行うセキュリティモデル。	社内ネットワークからのアクセスでも、ユーザーID、端末状態、アクセス先などを評価する。	**Never Trust, Always Verify** が代表的な考え方。継続的な評価、最小権限、マイクロセグメンテーションなどが重要。
10	**プライバシー・バイ・デザイン（Privacy by Design：PbD）**	システムやサービスの企画・設計段階から、個人情報保護の仕組みを組み込む考え方。	アプリケーション設計時に、収集する個人情報を最小化し、保存期間や削除方法を定義する。	**事後対応ではなく設計段階からプライバシーを保護する。** データ最小化やPrivacy by Defaultとの関係を理解する。
11	**責任共有モデル（Shared Responsibility Model）**	クラウドサービスのセキュリティ責任を、クラウド事業者と利用者で分担するモデル。	IaaSでは事業者が物理インフラを保護し、利用者がOS設定やアプリケーションなどを管理する。	**IaaS・PaaS・SaaSで責任分担が異なる。** クラウドを利用しても、利用者の責任がなくなるわけではない。
12	**SASE（Secure Access Service Edge）**	ネットワーク機能とセキュリティ機能をクラウド中心のサービスとして統合・提供するアーキテクチャ。	SD-WAN、SWG、CASB、ZTNA、FWaaSなどを組み合わせ、拠点やリモートユーザーのアクセスを保護する。	**ネットワークとセキュリティの統合**が特徴。ゼロトラストとの関係や、SSEとの違いを理解する。

CISSP試験対策｜混同しやすい概念

1. セキュアデフォルトとフェイルセキュア

- **セキュアデフォルト**：初期状態を安全にすること。
- **フェイルセキュア**：障害が発生してもセキュリティを維持すること。

覚え方

- Secure Defaults = 初期設定
- Fail Secure = 障害発生時

2. Trust but Verifyとゼロトラスト

- **Trust but Verify**：一定の信頼を前提としながら検証する。
- **Zero Trust**：ネットワーク上の位置などを理由に暗黙の信頼を与えず、アクセスを明示的に検証する。

覚え方

- Trust but Verify = 信頼するが確認する
- Zero Trust = 暗黙には信頼せず検証する

3. 最小権限と職務の分離

- **最小権限**：一人に与える権限を必要最小限にする。
- **職務の分離**：重要な業務を一人だけで完結できないようにする。

覚え方

- Least Privilege = 権限の最小化
- Separation of Duties = 職務の分割

CISSP試験で意識すること

これらの設計原則は、単純な用語の暗記だけでなく、「どのセキュリティ原則を適用すれば、リスクを最も適切に低減できるか」という判断問題で重要になる。

特に以下の観点を意識する。

1. **予防的な対策を優先する**
2. **設計段階からセキュリティを組み込む**
3. **単一の防御策に依存しない**
4. **必要最小限の権限を付与する**
5. **障害発生時のセキュリティと安全性を考慮する**

3.2 セキュリティモデルの基本概念を理解する

1. セキュリティモデルの比較

No.	モデル	主な目的	重要な概念	CISSP試験でのポイント
1	**Bell-LaPadula（ベル・ラパドゥラ）**	機密性（Confidentiality）の保護	Simple Security Property、*-Property、Strong *-Property	No Read Up / No Write Down
2	**Biba（ビバ）**	完全性（Integrity）の保護	Simple Integrity Property、*-Integrity Property、Invocation Property	No Read Down / No Write Up
3	**Brewer-Nash（ブリューワー・ナッシュ）**	利益相反（Conflict of Interest）の防止	Chinese Wall、動的アクセス制御、利益相反クラス	過去のアクセス履歴によってアクセス権限が変化する
4	**Clark-Wilson（クラーク・ウィルソン）**	商用システムにおける完全性の確保	CDI、UDI、TP、IVP、職務の分離	正当な処理手順と職務分離によってデータの完全性を維持する

No.	モデル	主な目的	重要な概念	CISSP試験でのポイント
5	**Graham-Denning（グラハム・デニング）**	アクセス権限の安全な管理	8つの基本的な保護ルール	オブジェクト・主体・アクセス権限の生成、削除、管理
6	**Harrison-Ruzzo-Ullman（HRU）**	アクセス制御システムの安全性の分析	Access Control Matrix、Safety Problem	権限が不適切に漏洩・付与され得るかを分析する

2. Bell-LaPadulaモデル

概要

Bell-LaPadulaモデルは、**情報の機密性（Confidentiality）を保護するためのセキュリティモデル**。

主に軍事機関や政府機関など、情報の機密区分が明確な環境を想定している。

情報と利用者にセキュリティレベルを設定し、情報が低い機密レベルへ漏洩することを防止する。

重要な概念

概念	説明	具体例
単純セキュリティ属性（Simple Security Property）	**No Read Up**。主体は、自分より高い機密レベルの情報を読み取れない。	Secret権限の利用者はTop Secret情報を閲覧できない。
スター属性（*-Property）	**No Write Down**。主体は、自分より低い機密レベルへ情報を書き込めない。	Top Secret権限の利用者はSecret領域へ情報を書き込めない。
** 強スター属性（Strong -Property）*	主体は、原則として自分と同じセキュリティレベルのオブジェクトにのみ読み書きできる。	Secret権限の利用者はSecretレベルの情報にのみ読み書きできる。

セキュリティレベルの例

Top Secret > Secret > Confidential > Unclassified

具体例

Secret権限を持つ利用者の場合：

操作	対象	許可
読み取り	Top Secret	❌
読み取り	Secret	✅
読み取り	Confidential	✅

操作	対象	許可
書き込み	Top Secret	✅
書き込み	Secret	✅
書き込み	Confidential	❌

※上記は単純セキュリティ属性と通常のスター属性を適用した場合。実際の許可にはカテゴリや裁量的アクセス制御などの追加条件も関係する。

CISSP試験でのポイント

- **機密性を保護するモデル**
- No Read Up
- No Write Down
- 高い機密レベルから低い機密レベルへの情報漏洩を防止する
- 強スター属性は通常のスター属性より厳格
- データの完全性を直接保証するモデルではない

---

3. Bibaモデル

概要

Bibaモデルは、**情報の完全性（Integrity）を保護するためのセキュリティモデル。**

信頼性の低い情報が、信頼性の高い情報に悪影響を与えることを防止する。

Bell-LaPadulaが機密性を重視するのに対し、Bibaは完全性を重視する。

重要な概念

概念	説明	具体例
単純完全性属性（Simple Integrity Property）	**No Read Down**。主体は、自分より低い完全性レベルの情報を読み取れない。	高信頼システムが低信頼データを直接読み取ることを禁止する。
スター完全性属性（*-Integrity Property）	**No Write Up**。主体は、自分より高い完全性レベルの情報を書き換えられない。	一般ユーザーが高信頼の基幹データを書き換えることを禁止する。
呼び出し属性（Invocation Property）	主体は、自分より高い完全性レベルの主体を呼び出せない。	低信頼プロセスが高信頼プロセスを直接呼び出すことを禁止する。

完全性レベルの例

High Integrity > Medium Integrity > Low Integrity

具体例

Medium Integrityの主体の場合：

操作	対象	許可
読み取り	High Integrity	✅
読み取り	Medium Integrity	✅
読み取り	Low Integrity	❌
書き込み	High Integrity	❌
書き込み	Medium Integrity	✅
書き込み	Low Integrity	✅

※厳格なBibaモデルの読み書き属性を適用した場合。

CISSP試験でのポイント

- **完全性を保護するモデル**
- No Read Down
- No Write Up
- 低信頼データによる高信頼データの汚染を防止する
- Bell-LaPadulaとは読み書きの制約が逆になる
- 実際のシステムでは、完全性と機密性の要求に応じて制御を組み合わせる

---

4. Brewer-Nashモデル（Chinese Wallモデル）

概要

Brewer-Nashモデルは、**利益相反（Conflict of Interest）を防止するためのセキュリティモデル。**

利用者が過去にアクセスした情報に応じて、今後アクセスできる情報が動的に変化する。

主に金融機関、コンサルティング会社、監査法人などで想定される。

重要な概念

概念	説明	具体例
利益相反クラス（Conflict of Interest Class）	利益相反が生じる可能性のある企業や情報を同じグループに分類する。	競合する銀行A社と銀行B社を同じ利益相反クラスに分類する。
企業データセット（Company Dataset）	特定企業に関係する情報の集合。	A銀行の財務情報、経営計画、顧客情報。
動的アクセス制御	過去のアクセス履歴によって、アクセス可能な情報が変化する。	A銀行の機密情報を閲覧した利用者は、競合するB銀行の機密情報を閲覧できなくなる。

概念	説明	具体例
Chinese Wall	利益相反が発生する情報同士を論理的に隔離する。	同じ担当者が競合企業双方の機密情報を利用できないようにする。

具体例

コンサルタントが以下の企業情報を扱う場合：

- A銀行
- B銀行
- C自動車メーカー

A銀行とB銀行は同じ利益相反クラスに属する。

コンサルタントがA銀行の機密情報にアクセスした場合：

対象	アクセス
A銀行の情報	✅
B銀行の機密情報	❌
C自動車メーカーの情報	✅

※他のアクセス制御条件を満たす場合。公開情報などのサニタイズされた情報は別途扱われる。

CISSP試験でのポイント

- **利益相反の防止**
- 過去のアクセス履歴が重要
- アクセス権限が動的に変化する
- 金融・監査・コンサルティング業務との関連性が高い
- 単純な機密レベルの上下関係ではなく、競合関係によってアクセスを制御する

---

5. Clark-Wilsonモデル

概要

Clark-Wilsonモデルは、**商用システムにおけるデータの完全性を保護するためのセキュリティモデル**。

データを自由に変更させるのではなく、認可された正当な処理手順を通じて変更させる。

特に以下の考え方を重視する。

- 適切なトランザクション処理
- 職務の分離
- データの整合性検証
- 監査可能性

重要な概念

概念	正式名称	説明	具体例
CDI	Constrained Data Item	完全性を維持する必要がある、統制されたデータ。	銀行の口座残高、会計台帳。
UDI	Unconstrained Data Item	まだ完全性が保証されていないデータ。	外部から入力された取引データ。
TP	Transformation Procedure	CDIを正当な方法で変更するための認可された処理。	銀行の送金処理プログラム。
IVP	Integrity Verification Procedure	CDIが完全性制約を満たしていることを検証する処理。	会計データの整合性チェック。
職務の分離	Separation of Duties	重要な処理を一人で完結できないようにする。	送金申請者と承認者を分離する。
認証・認可	Authentication / Authorization	正当な利用者だけが許可されたTPを実行できるようにする。	承認権限を持つ担当者だけが送金を承認できる。

具体例

銀行の送金処理を考える。

1. 利用者が送金情報を入力する。
2. 入力情報はUDIとして扱われる。
3. 認可されたTPが入力内容を検証する。
4. 必要な承認を経てTPがCDIを更新する。
5. IVPによってCDIの整合性を検証する。

このように、重要データを直接編集させず、認可された処理を経由して変更する。

CISSP試験でのポイント

- **完全性を重視するモデル**
- 商用システムや金融システムとの関連性が高い
- CDI、UDI、TP、IVPを区別する
- Well-Formed Transactions（適切に形成されたトランザクション）
- Separation of Duties（職務の分離）
- データを直接変更するのではなく、認可されたTPを通じて変更する
- Bibaとは異なり、完全性レベルの上下関係よりも処理手順や統制を重視する

---

6. Graham-Denningモデル

概要

Graham-Denningモデルは、**主体・オブジェクト・アクセス権限を安全に管理するためのアクセス制御モデル**。

アクセス制御システムにおける権限の生成、削除、付与、移転などを定義する。

重要な概念

概念	説明
主体（Subject）	アクセスを要求するユーザーやプロセス。
オブジェクト（Object）	アクセス対象となるファイル、データ、リソースなど。
アクセス権限（Access Rights）	読み取り、書き込み、実行などの許可。
アクセス制御マトリックス	主体とオブジェクトの間に存在するアクセス権限を表現する仕組み。

8つの基本的な保護ルール

No.	ルール	説明
1	Create Object	新しいオブジェクトを生成する。
2	Delete Object	オブジェクトを削除する。
3	Create Subject	新しい主体を生成する。
4	Delete Subject	主体を削除する。
5	Read Access Right	アクセス権限を読み取る。
6	Grant Access Right	アクセス権限を付与する。
7	Delete Access Right	アクセス権限を削除する。
8	Transfer Access Right	アクセス権限を移転する。

具体例

システム管理者が新しいユーザーを作成し、特定のファイルへの読み取り権限を付与する。

その後、担当業務が変更されたため、アクセス権限を削除する。

このような権限管理操作を、安全に実行するためのルールを定義する。

CISSP試験でのポイント

- **アクセス権限の管理に重点を置くモデル**
- 主体とオブジェクトの生成・削除
- アクセス権限の付与・削除・移転
- 8つの基本的な保護ルール
- 機密性や完全性そのものより、アクセス制御の管理操作に注目する

---

7. Harrison-Ruzzo-Ullmanモデル（HRU）

概要

HRUモデルは、**アクセス制御システムにおける権限の安全性を分析するためのモデル。**

アクセス制御マトリックスを基礎とし、権限の追加・削除や主体・オブジェクトの生成・削除によって、システムの状態がどのように変化するかを分析する。

重要な概念

概念	説明	具体例
Access Control Matrix	主体とオブジェクトのアクセス権限を表現するマトリックス。	ユーザーAはファイルXを読み取り可能、ユーザーBは書き込み可能。
Safety Problem	特定の権限が、将来ある主体に付与され得るかを判定する問題。	一般ユーザーが将来、管理者権限を取得できる状態になるか。
権限の追加・削除	アクセス権限を変更する操作。	ユーザーに書き込み権限を付与する。
主体・オブジェクトの生成・削除	アクセス制御システムの構成要素を変更する操作。	新しいユーザーやファイルを作成する。

Safety Problemとは

アクセス制御システムにおいて、ある権限が本来想定していない主体に付与される可能性があるかを判定する問題。

例えば、一般ユーザーが複数の権限変更操作を経て、機密ファイルへの書き込み権限を取得できるかを分析する。

一般的なHRUモデルでは、Safety Problemは決定不能（Undecidable）であることが知られている。

つまり、任意のHRUシステムについて、安全性を常に判定できる一般的なアルゴリズムは存在しない。

CISSP試験でのポイント

- **アクセス制御システムの安全性を分析するモデル**
- Access Control Matrix
- Safety Problem
- 権限の追加・削除
- 主体・オブジェクトの生成・削除
- 一般的なSafety Problemは決定不能
- Graham-Denningと関連するが、HRUは権限変更による安全性の分析を重視する

---

8. CISSP試験対策｜モデルの違い

比較対象	違い
Bell-LaPadula vs Biba	Bell-LaPadulaは機密性、Bibaは完全性を保護する。読み書きの制約が逆になる。
Biba vs Clark-Wilson	Bibaは完全性レベルによるアクセス制御、Clark-Wilsonは正当な処理手順と職務の分離を重視する。
Brewer-Nash vs Bell-LaPadula	Brewer-Nashは利益相反とアクセス履歴、Bell-LaPadulaは機密区分による情報漏洩防止を重視する。

比較対象	違い
Graham-Denning vs HRU	Graham-Denningは権限管理操作、HRUは権限変更による安全性の分析を重視する。

9. 最重要暗記事項

モデル	覚えるべきキーワード
Bell-LaPadula	**Confidentiality / No Read Up / No Write Down**
Biba	**Integrity / No Read Down / No Write Up**
Brewer-Nash	**Conflict of Interest / Chinese Wall / Dynamic Access Control**
Clark-Wilson	**Integrity / CDI / UDI / TP / IVP / Separation of Duties**
Graham-Denning	**Access Rights Management / 8 Protection Rules**
HRU	**Access Control Matrix / Safety Problem / Undecidability**

10. 試験での判断方法

問題文に以下の表現が出てきた場合、対応するモデルを検討する。

- 「機密情報が低い機密レベルへ漏洩することを防止」→ Bell-LaPadula
- 「信頼性の低いデータによる高信頼データの汚染を防止」→ Biba
- 「競合企業間の利益相反を防止」→ Brewer-Nash
- 「正当なトランザクションと職務分離によって完全性を維持」→ Clark-Wilson
- 「主体・オブジェクト・権限の生成や削除を管理」→ Graham-Denning
- 「将来的な権限漏洩の可能性や安全性を分析」→ HRU

3.3 システムセキュリティ要件に基づくコントロールの選択

1. セキュリティエンジニアリングの基本概念と関連標準

No.	用語・標準	概要・定義	重要な概念・特徴	具体例	CISSP試験でのポイント
1	**セキュリティエンジニアリング（Security Engineering）**	システムのライフサイクル全体を通じて、セキュリティ要件を定義し、適切な対策を設計・実装・検証する工学的アプローチ。	セキュリティ要件定義、リスク分析、セキュア設計、検証、継続的改善。	新規システムの設計段階で脅威分析を実施し、暗号化やアクセス制御を設計に組み込む。	**Security by Design**が重要。セキュリティを運用段階で追加するのではなく、設計段階から組み込む。

No.	用語・標準	概要・定義	重要な概念・特徴	具体例	CISSP試験でのポイント
2	**セキュリティエンジニアリング標準（Security Engineering Standards）**	システムの安全性や信頼性を確保するため、セキュリティエンジニアリングの活動やプロセスを体系化した標準・ガイドライン。	標準化、ライフサイクル管理、要件の明確化、プロセスの一貫性。	組織がNISTやISOの標準を参考に、システム開発時のセキュリティ設計基準を策定する。	**標準をそのまま適用するのではなく、組織のリスク・業務要件に合わせて調整することが重要。**
3	**INCOSE Systems Engineering Handbook（インコウシー／インコース）**	INCOSE（International Council on Systems Engineering）が発行する、システムズエンジニアリングの実践方法を体系化したハンドブック。	システムライフサイクル、要求分析、アーキテクチャ設計、検証、妥当性確認。	大規模システム開発において、要件定義から設計・実装・検証までの活動を体系的に管理する。	**システムズエンジニアリング全体の実践指針。** セキュリティ専用の標準ではなく、システム開発全体を対象とする。
4	**NIST SP 800-160（Systems Security Engineering）**	NISTが発行する、システムライフサイクル全体にセキュリティエンジニアリングを組み込むためのガイドライン。	セキュリティ要件、システムライフサイクル、セキュリティ設計、信頼性、保証。	システムの企画段階からセキュリティ要件を定義し、設計・実装・運用まで一貫して管理する。	**セキュリティをシステムエンジニアリングに統合することが目的。** NIST SP 800-53のセキュリティコントロール集とは役割が異なる。
5	**ISO/IEC 15026（Systems and Software Engineering — Systems and Software Assurance）**	システムおよびソフトウェアの保証（Assurance）に関する国際規格シリーズ。	Assurance Case、保証のための証拠、保証プロセス、信頼性の説明。	重要システムについて、セキュリティ要件が満たされていることを試験結果やレビュー記録などの証拠で示す。	**保証（Assurance）が中心。** 単に安全だと主張するのではなく、根拠となる証拠によって説明する。
6	**ISO/IEC/IEEE 15288（Systems and Software Engineering — System Life Cycle Processes）**	システムのライフサイクル全体に適用するプロセスを定義した国際規格。	技術プロセス、技術管理プロセス、合意プロセス、組織のプロジェクト実現プロセス。	システム開発において、要件定義、設計、実装、検証、運用、保守、廃棄までのプロセスを体系的に管理する。	**システムライフサイクル全体のプロセスを定義する規格。** セキュリティ専用ではなく、システムエンジニアリング全体を対象とする。

No.	用語・標準	概要・定義	重要な概念・特徴	具体例	CISSP試験でのポイント
7	**セキュリティエンジニアリングプロセス（Security Engineering Process）**	セキュリティ要件の特定から、設計、実装、検証、運用に至るまで、セキュリティを体系的に確保するための一連の活動。	リスク分析、要件定義、セキュリティ設計、コントロールの選択、検証、継続的改善。	システム開発時にリスクを分析し、必要なセキュリティ対策を選定して、実装後に有効性を検証する。	**リスクと要件に基づいてコントロールを選択することが重要。** 技術的な対策だけでなく、組織の目的や業務要件も考慮する。

2. 関連標準の違い【補足】

標準	主な対象	覚えるべきポイント
INCOSE Handbook	システムズエンジニアリング全般	システム開発全体の実践指針
NIST SP 800-160	システムセキュリティエンジニアリング	ライフサイクル全体へのセキュリティ統合
ISO/IEC 15026	システム・ソフトウェア保証	証拠に基づく保証（Assurance）
ISO/IEC/IEEE 15288	システムライフサイクルプロセス	ライフサイクル全体のプロセス定義

3. 略語と読み方【補足】

略語	読み方	正式名称
INCOSE	インコウシー（インコースとも表記される）	International Council on Systems Engineering
NIST	ニスト	National Institute of Standards and Technology
ISO	アイエスオー／イソ	International Organization for Standardization
IEC	アイイーシー	International Electrotechnical Commission
IEEE	アイトリプルイー	Institute of Electrical and Electronics Engineers
SP	エスピー	Special Publication

- システムのライフサイクルに関するプロセスは、大きく4つに分類される。

分類	役割	覚え方
技術プロセス	システムを作る	何を、どう作るか
技術管理プロセス	開発活動を管理する	計画どおり進んでいるか
プロセスの有効化	組織として開発を支える	開発できる環境を整える
契約プロセス	顧客と供給者の関係を管理する	買う側・売る側

1. 技術プロセス

目的：システムの構想から設計・実装・検証までを実施する。

プロセス	シンプルな解説	覚え方
ビジネス及びミッション分析	ビジネス上の課題や目的を明確にする	なぜ作る？
利害関係者のニーズと要件定義	利用者や関係者が求めることを整理する	何が欲しい？
システム要件定義	ニーズを具体的なシステム要件に落とし込む	何が必要？
アーキテクチャ定義	システム全体の構造や構成要素を決める	どんな構造？
設計定義	各構成要素の詳細な仕様を決める	具体的にどう作る？
システム分析	設計案などを分析し、要件を満たせるか評価する	この方式で大丈夫？
実装	設計に基づきシステムの構成要素を作る	実際に作る
インテグレーション	個別の構成要素を組み合わせる	つなぎ合わせる
検証	成果物が定められた要件・仕様を満たすか確認する	正しく作れた？

試験対策：混同しやすいポイント

① ニーズとシステム要件の違い

- ニーズ：利用者が実現したいこと
- システム要件：それを実現するためにシステムが満たす条件

例：

- ニーズ：「安全にログインしたい」
- システム要件：「多要素認証を実装する」

② アーキテクチャと設計の違い

- アーキテクチャ：システム全体の構造
- 設計：個々の構成要素の具体的な仕様

例：

- アーキテクチャ：Web・AP・DBの3層構造
- 設計：DBのテーブル構造やAPI仕様

③ 検証（Verification）と妥当性確認（Validation）

- 検証：仕様どおりに作ったか
- 妥当性確認：利用者の目的に合ったものを作ったか

覚え方：

- Verification = 正しく作ったか
- Validation = 正しいものを作ったか

2. 技術管理プロセス

目的：プロジェクトの進行・リスク・品質などを管理する。

プロセス	シンプルな解説	覚え方
プロジェクト管理	スケジュール・予算・人員などを計画する	計画を立てる
プロジェクトアセスメントおよびコントロール	進捗を評価し、必要に応じて是正する	計画との差を直す
意思決定管理	複数の選択肢を評価して最適な判断を行う	どれを選ぶ？
リスク管理	リスクを特定・分析・評価・対応する	何が危ない？
構成管理	システムの構成や変更履歴を管理する	何が変わった？
情報管理	必要な情報の作成・保管・共有・保護を管理する	情報をどう扱う？
測定	進捗や品質などを数値で把握する	数字で測る
品質保証	プロセスや成果物が品質基準に適合するよう保証する	品質を守れている？

試験対策：混同しやすいポイント

① プロジェクト管理とアセスメント・コントロール

- プロジェクト管理：計画を立てる
- アセスメント・コントロール：実績を評価して修正する

例：

- 管理：「開発期間は6か月」
- アセスメント：「進捗が2週間遅れている」
- コントロール：「要員を追加して遅延を是正する」

② 構成管理と変更管理

- 構成管理：構成要素・バージョン・変更状態を把握する
- 変更管理：変更の申請・評価・承認などを統制する

※変更管理は構成管理と密接に関連する。

③ 測定と品質保証

- 測定：品質や進捗を数値化する
- 品質保証：品質基準を満たす仕組みや活動を確保する

3. プロセスの有効化

目的：組織全体として、プロジェクトを継続的に実施できる環境を整える。

プロセス	シンプルな解説	覚え方
ライフサイクルモデル管理	開発・運用などの標準的な進め方を整備する	開発の型を決める
インフラ管理	開発や運用に必要な設備・ツール・環境を整える	作業環境を用意
ポートフォリオ管理	複数のプロジェクトを評価し、投資や優先順位を決める	どの案件をやる？
人事管理	必要な人材の確保・育成・配置を行う	人を整える
品質管理	組織全体の品質方針・基準・仕組みを整備する	品質のルールを作る
知識管理	ノウハウや教訓を蓄積・共有・再利用する	経験を資産にする

試験対策：混同しやすいポイント

① プロジェクト管理とポートフォリオ管理

- プロジェクト管理：個別案件を成功させる
- ポートフォリオ管理：組織全体で案件の優先順位を決める

例：

- プロジェクト管理：「A案件を納期までに完了させる」
- ポートフォリオ管理：「A案件よりB案件に投資する」

② 品質保証と品質管理

- 品質保証：個別のプロジェクトや成果物が品質要求に適合するようにする
- 品質管理：組織として品質方針や品質マネジメントの仕組みを整える

※ここでの品質管理は、組織レベルの 「Quality Management」を指す。

③ 知識管理と情報管理

- 情報管理：情報の取り扱いを管理する
- 知識管理：経験・ノウハウを組織の資産として活用する

例：

- 情報管理：設計書のアクセス権限を設定する
- 知識管理：障害対応の教訓を他案件に展開する

4. 契約プロセス

目的：システムの取得者と供給者の契約関係を管理する。

プロセス	シンプルな解説	覚え方
取得プロセス	必要な製品やサービスを外部から調達する	買う側
供給プロセス	契約に基づき製品やサービスを提供する	売る側

試験対策

取得プロセス（Acquisition）

- 調達要件を定義する
- 供給者を評価・選定する
- 契約を締結する
- 納入物を受け入れる

供給プロセス（Supply）

- 顧客の要求を確認する
- 提案・契約を行う
- 製品やサービスを提供する
- 契約上の義務を履行する

覚え方：

- Acquisition = Buyer（買い手）
- Supply = Supplier（売り手）

5. CISSP試験直前の暗記表

キーワード	対応するプロセス
なぜ必要なのか	ビジネス及びミッション分析
利用者が何を求めるか	利害関係者のニーズと要件定義
システムが満たす条件	システム要件定義
システム全体の構造	アーキテクチャ定義
詳細な仕様	設計定義
技術的な実現可能性の評価	システム分析
実際に作る	実装
構成要素を結合する	インテグレーション
仕様への適合を確認する	検証
計画を立てる	プロジェクト管理
進捗を評価・是正する	アセスメント・コントロール
選択肢を評価する	意思決定管理
不確実性に対応する	リスク管理
バージョン・構成を追跡する	構成管理
情報を保管・共有・保護する	情報管理
KPIなどを数値化する	測定
品質要求への適合を保証する	品質保証

キーワード	対応するプロセス
開発手法の標準化	ライフサイクルモデル管理
開発環境の整備	インフラ管理
複数案件の投資判断	ポートフォリオ管理
人材の確保・育成	人事管理
組織の品質方針・仕組み	品質管理
ノウハウの蓄積・再利用	知識管理
調達する側	取得
提供する側	供給

3-4. 情報システムのセキュリティ能力を理解する

1. セキュリティモデルの基本概念

セキュリティモデルとは、システム内の情報やリソースを保護するための理論的な枠組み。

基本用語

用語	意味	覚え方
サブジェクト（Subject）	操作を実行する主体（プロセスなど）	操作する側
オブジェクト（Object）	操作の対象となる情報やリソース	操作される側
アクセス制御	操作の許可・拒否を判断する仕組み	操作してよいか

※同じプロセスでも、他のプロセスから操作される場合はオブジェクトになる。

2. ステートマシンモデル（State Machine Model）

概要

システムが安全な状態から安全な状態へ遷移することで、セキュリティを維持するモデル。

基本原則

- システムには複数の状態（State）が存在する。
- 初期状態が安全であることが前提。
- すべての状態遷移で安全性を維持する。

具体例

ファイルへのアクセス制御

1. 権限のないユーザーはファイルを閲覧できない。
2. ユーザーがアクセスを要求する。
3. システムがアクセス権限を確認する。

4. 権限がなければ拒否し、安全な状態を維持する。

覚え方

「安全 → 変化 → 安全」

3. 情報フローモデル（Information Flow Model）

概要

情報がどこからどこへ流れるかを制御するモデル。

基本原則

- 情報の流れる方向や経路を制御する。
- セキュリティポリシーに違反する情報の流れを禁止する。
- 直接的なアクセスだけでなく、情報の移動にも注目する。

具体例

機密性を重視するシステムの場合

- 機密領域 → 公開領域：原則禁止
- 公開領域 → 機密領域：ポリシーに従って許可

※許可される情報の流れは、採用するセキュリティポリシーによって異なる。

覚え方

「情報がどこへ流れるか」

4. 非干渉モデル（Noninterference Model）

概要

高いセキュリティレベルの活動が、低いセキュリティレベルから観測できる結果に影響しないようにするモデル。

基本原則

- 高機密領域の活動が低機密領域の観測結果に影響しない。
- 低機密側から高機密側の情報を推測できないようにする。
- 情報の直接的な移動だけでなく、間接的な影響も考慮する。

具体例

機密システムと一般システム

1. 機密システムで秘密の処理を実行する。
2. 一般システムから処理結果や動作の変化を観測する。
3. その観測結果から秘密の処理内容を推測できないようにする。

覚え方

「上の世界の動きを下に悟らせない」

5. リングモデル（Ring Model）

概要

システムを複数の特権レベルに分割し、重要な機能へのアクセスを制限する保護モデル。

基本原則

- システムを複数の保護リングに分割する。
- 内側ほど高い特権を持つ。
- 外側ほど利用できる特権が制限される。

代表的な4つのリング

リング	特権レベル	主な用途・イメージ
Ring 0	最高	OSカーネル
Ring 1	高	OSサービスなど
Ring 2	中	デバイスドライバなど
Ring 3	最低	一般アプリケーション

※Ring 1・2の用途は概念的な例。 実際のOSではRing 0とRing 3を中心に利用することが多い。

覚え方

「内側ほど偉い」

6. CISSP試験対策：4モデル比較

モデル	保護・制御するもの	基本的な考え方	覚え方
ステートマシン	システムの安全な状態	安全な状態遷移を維持	安全 → 変化 → 安全
情報フロー	情報の流れる方向	不正な情報の流れを禁止	情報はどこへ流れる？
非干渉	高機密側から低機密側への影響	高機密側の活動を低機密側に悟らせない	上の動きを下に悟らせない
リング	システムの特権レベル	特権を階層的に分離	内側ほど偉い

セキュリティアーキテクチャの構成要素

概要

セキュアなシステムを実現するために、CPU・OS・メモリ・アプリケーションなどが備えるべきセキュリティ機能。

主要な構成要素

用語	概要	具体例・覚え方
特権モード命令	OSなどの特権モードでのみ実行できる命令	メモリ管理設定の変更
プロセッサ状態	CPUの現在の実行状態や特権レベルを示す	ユーザーモード／カーネルモード
メモリ管理	プロセスごとのメモリ領域を管理・保護する	仮想メモリ、ページング
分離レイヤー	システムの構成要素を分離し、相互干渉を制限する	OSとアプリケーションの分離
データ空間とコード空間の分離	プログラムの命令とデータを分離して保護する	NXビット、DEP
ファイルシステム属性	ファイルのアクセス権限や保護属性を管理する	ACL、読み取り専用属性
セキュリティカーネル	セキュリティポリシーを強制する中核的な仕組み	リファレンスモニターの実装
アクセス制御	主体によるリソースへの操作を許可・拒否する	DAC、MAC、RBAC
仮想マシン	物理資源を仮想化し、独立した実行環境を提供する	VMware、Hyper-V
サンドボックス環境	プログラムを制限された環境で実行する	ブラウザのサンドボックス
暗号システム	暗号技術を利用して情報を保護する	AES、RSA、TLS

CISSP試験で混同しやすいポイント

比較対象	違い
特権モード命令／プロセッサ状態	特権モード命令は「実行できる命令」、プロセッサ状態は「CPUの実行状態」
メモリ管理／分離レイヤー	メモリ管理は「メモリ領域の保護」、分離レイヤーは「構成要素間の隔離」
セキュリティカーネル／アクセス制御	セキュリティカーネルは「ポリシーを強制する中核」、アクセス制御は「操作の許可・拒否」
仮想マシン／サンドボックス	仮想マシンは「仮想化された実行環境」、サンドボックスは「制限された実行環境」

最重要ポイント

セキュリティアーキテクチャは、複数の保護機能を組み合わせてシステム全体の安全性を実現する。

- CPU：特権命令や実行モードを制御する
- メモリ：プロセス間の不正なアクセスを防ぐ
- OS：アクセス制御やセキュリティポリシーを強制する
- 実行環境：仮想化やサンドボックスによって隔離する

- データ：暗号化などによって情報を保護する

※セキュリティカーネルは、OSカーネル全体と必ずしも同義ではない。 ※仮想マシンやサンドボックスも、単独で完全な安全性を保証するものではない。

トラステッドプラットフォームモジュール（TPM）

- **概要**：暗号鍵を安全に保護し、システムの信頼性を確保するためのハードウェアベースのセキュリティ機能。
- **主な用途**：暗号鍵の保護、BitLockerによるディスク暗号化の支援、セキュアブートなどと連携した起動状態の検証。
- **覚え方**：**「暗号鍵を守り、システムの信頼性を支える」**

3.5 セキュリティアーキテクチャ、設計、ソリューション要素の脆弱性を評価・低減する

学習の目的

システムアーキテクチャーごとの特徴、固有の脆弱性、低減策を理解し、組織のリスクに応じた適切なセキュリティ対策を判断する。

CISSPの基本的な考え方

- セキュリティ対策はシステムの目的・特性・制約に応じて選択する。
- 機密性・完全性・可用性（CIA）の優先順位はシステムによって異なる。
- セキュリティ技術の導入自体ではなく、リスクの適切な管理が目的である。
- 資産管理、リスク評価、アクセス制御、監視、変更管理、継続的改善は多くのアーキテクチャーに共通する。

1. クライアントベースシステム（Client-Based Systems）

概要

利用者が操作するPC、ノートPC、ワークステーション、POS端末などのクライアントを中心としたシステム。

- サーバーにアクセスしてサービスやデータを利用する。
- シンクライアント（処理の多くをサーバー側に依存する端末）とシッククライアント（端末側で多くの処理を実行する端末）がある。
- 多数の端末が存在し、更新・交換・移動が頻繁に発生する。

主な脆弱性と低減策

脆弱性・リスク	概要	主な低減策
OS・アプリケーションの脆弱性	未修正の脆弱性が攻撃に悪用される	パッチ管理、脆弱性管理
マルウェア感染	利用者の操作や不正ファイルを通じて感染する	EDR、マルウェア対策、利用者教育
端末の紛失・盗難	保存データや認証情報が流出する	ディスク暗号化、MDM、アクセス制御

脆弱性・リスク	概要	主な低減策
管理対象の増大	多数の端末の設定や更新にばらつきが生じる	集中管理、構成標準化
侵害端末からの横展開	攻撃者が他の端末やサーバーへ侵入する	ネットワーク分離、最小権限、監視

※上表の低減策には、教材内容の整理に加え、実務上の補足を含む。

CISSP重要ポイント

クライアントベースシステムの中心課題は、多数のエンドポイントを継続的に管理・保護すること。

- EDR（Endpoint Detection and Response）：端末上の不審な挙動を検知・調査・対応する仕組み。
- MDM（Mobile Device Management）：モバイル端末などを集中管理する仕組み。
- 端末の安全性だけでなく、侵害後の横展開防止も重要。

2. サーバーベースシステム（Server-Based Systems）

概要

Webサーバー、ファイルサーバー、DNSサーバーなど、特定のサービスを提供するシステム。

- オンプレミス、クラウド、ハイブリッド環境で利用される。
- 多数の利用者やシステムからアクセスされる。
- サービス停止が業務全体に影響する場合がある。

主な脆弱性と低減策

脆弱性・リスク	概要	主な低減策
OS・アプリケーションの脆弱性	サーバーの侵害や情報漏えいにつながる	パッチ管理、構成管理
更新作業による停止	稼働要件が厳しくパッチを適用しにくい	冗長化、計画的な変更管理
レガシーシステム	サポート終了などにより脆弱性が残る	更新・移行、代替的な保護策
不正アクセス	管理権限や重要データが狙われる	最小権限、管理者認証の強化
物理・ネットワーク攻撃	設備障害や通信攻撃により停止する	物理的保護、ネットワーク監視

CISSP重要ポイント

サーバーでは、セキュリティ対策と可用性の両立が重要。

特に、冗長化によってサービスを継続しながらパッチ適用や保守を実施できる設計が望ましい。

- 冗長化（Redundancy）：同じ機能を提供する構成要素を複数用意すること。
- フェイルオーバー（Failover）：障害時に予備系へ処理を切り替えること。

3. 暗号システム（Cryptographic Systems）

概要

暗号技術によって情報の機密性、完全性、真正性などを保護するシステム。

保護対象となるデータの状態は主に3つ。

データの状態	意味	主な保護方法
保存中（Data at Rest）	ディスクやデータベースに保存された情報	ストレージ暗号化
転送中（Data in Transit）	ネットワーク上を移動する情報	TLSなどによる通信保護
使用中（Data in Use）	メモリ上などで処理されている情報	メモリ保護、TEEなど

主なリスクと設計上の対策

※教材では独立した脆弱性・低減策の一覧よりも、暗号システムの設計原則が中心。

リスク	概要	対策
独自暗号方式の欠陥	安全性が十分に検証されていない	実績のある標準暗号を利用する
暗号鍵の漏えい	暗号方式が安全でも情報が復号される	適切な鍵管理
鍵配送の問題	共通鍵を安全に共有できない	公開鍵暗号や鍵共有技術を利用する
処理性能への影響	暗号化による計算負荷が生じる	用途に応じた暗号方式を選択する

CISSP重要ポイント

独自の暗号アルゴリズムを開発せず、十分に検証された標準技術を採用する。

- 共通鍵暗号：同じ鍵で暗号化・復号する。高速な大量データ処理に適する。
- 公開鍵暗号：公開鍵と秘密鍵を利用する。鍵共有や電子署名などに利用される。
- ハイブリッド暗号：共通鍵暗号と公開鍵技術を組み合わせる。
- TEE（Trusted Execution Environment）：通常の実行環境から隔離された保護領域。

暗号アルゴリズムの強度だけでなく、**鍵の生成・保管・配布・更新・廃棄**が重要。

4. データベースシステム（Database Systems）

概要

DBMS（Database Management System：データベース管理システム）を利用して大量のデータを保存・検索・更新するシステム。

- 機密性の高い情報を大量に保持する。
- 複数利用者からの同時アクセスが発生する。
- OS、ネットワーク、アプリケーション、DBMSの各層に脆弱性が存在する。

主な脆弱性と低減策

教材の脆弱性	概要	低減策
不正な入力	不適切な入力により意図しない処理が実行される	入力検証、パラメータ化クエリ
DoS	大量の要求などでデータベースが利用できなくなる	リソース制限、監視、冗長化
アクセス制御の回避	本来アクセスできない情報にアクセスされる	最小権限、RBAC、監査ログ

※低減策は教材の脆弱性に対応するCISSP・実務上の補足。

CISSP重要ポイント

データベースでは、入力制御・アクセス制御・可用性の確保が重要。

- SQLインジェクション：不正なSQLを入力に混入させ、意図しないデータベース操作を行わせる攻撃。
- パラメータ化クエリ：SQLの命令と入力値を分離する手法。
- RBAC（Role-Based Access Control）：役割に基づいてアクセス権限を割り当てる方式。

アプリケーション、DBMS、OS、ネットワークを組み合わせた多層防御が重要。

5. 産業制御システム（ICS：Industrial Control Systems）

概要

工場、発電所、水道、鉄道、パイプラインなど、物理的な設備や産業プロセスを監視・制御するシステム。

一般的なITシステムと異なり、**人命・設備の安全性、可用性、リアルタイム性**が特に重要となる。

主な構成要素

要素	役割	代表的な用途
SCADA	広域に分散した設備の監視・制御	電力網、パイプライン
DCS	工場などの設備内で分散制御を行う	化学プラント、製造設備
PLC	センサー入力に応じて機械を直接制御する	モーター、バルブ、製造ライン

- SCADA（Supervisory Control and Data Acquisition）：監視制御・データ収集システム。
- DCS（Distributed Control System）：分散制御システム。
- PLC（Programmable Logic Controller）：産業用の制御装置。

主な脆弱性と低減策

脆弱性・リスク	概要	主な低減策
セキュリティ機能の制約	組み込みOSなどに十分な保護機能がない	ネットワーク分離、代替的な保護策
長い製品寿命	10〜20年など長期間利用され、更新が難しい	計画的な更新、変更管理
DoSへの弱さ	通信や入力処理が攻撃に耐えられない	通信制限、監視
物理的な攻撃	遠隔地の無人設備が侵害される	物理的アクセス制御

脆弱性・リスク	概要	主な低減策
ベンダー依存	独自仕様により安全性評価が難しい	ベンダー評価、リスクアセスメント
変更による運用影響	パッチ適用などが設備停止につながる	事前検証、段階的な変更管理

CISSP重要ポイント

ICSでは、セキュリティ対策が物理的な安全性や設備の可用性を損なわないことが重要。

主な対策：

- ITネットワークとOTネットワークの分離
- ファイアウォールやアクセス制御
- 通信の保護
- ネットワーク監視と早期検知
- 厳格な変更管理
- 必要に応じたマイクロセグメンテーション

OT（Operational Technology）：物理的な設備やプロセスを監視・制御する技術。

覚え方：ICSは「安全に止めずに守る」。

6. 組み込みシステム（Embedded Systems）

概要

特定の機能を実行するため、機器内部に組み込まれたコンピューターシステム。

例：

- 自動車
- 航空機
- 医療機器
- 家電
- 産業機器
- セキュリティ機器

一般的なPCと異なり、処理能力、メモリ、電力、更新機能などに制約がある場合が多い。

主な脆弱性と低減策

教材の脆弱性	概要	主な低減策
プログラミングの欠陥	入力検証不足、メモリ管理不備など	セキュアコーディング、コード解析
Web管理画面の脆弱性	管理インターフェースから侵入される	更新、アクセス制限
弱い認証情報	初期パスワードや固定認証情報が悪用される	強固な認証、認証情報の管理
不適切な暗号	古い暗号方式や誤った実装が使われる	標準暗号、適切な鍵管理

教材の脆弱性	概要	主な低減策
リバースエンジニアリング	ファームウェアを解析される	コード難読化、設計上の保護
マルウェア	機器の動作が改ざんされる	更新、実行制御、ネットワーク分離
盗聴・リプレイ攻撃	通信内容が取得・再利用される	通信暗号化、リプレイ防止

※ネットワーク分離や実行制御など、一部は教材の対策を補う実務上の対策。

重要用語

- ファームウェア：機器を制御するために組み込まれたソフトウェア。
- NVD（National Vulnerability Database）：公開された脆弱性情報を整理するデータベース。
- リバースエンジニアリング：プログラムや機器を解析して内部構造を調べること。
- リプレイ攻撃：過去の正当な通信を再送して不正な処理を行わせる攻撃。
- Stuxnet：産業制御システムを標的としたことで知られるマルウェア。
- SAST：ソースコードなどを実行せずに解析する静的テスト。
- DAST：実行中のアプリケーションに対して行う動的テスト。

CISSP重要ポイント

組み込みシステムでは、機器の制約と更新の難しさを前提にリスクを管理する。

- すべての機器に一般的なエンドポイント保護製品を導入できるわけではない。
- 更新できない機器には代替的な保護策が必要。
- コード難読化だけでは安全性を保証できない。
- ベンダーのサポート期間や更新提供能力も評価する。

7. IoT（Internet of Things）

概要

ネットワークに接続されたセンサー、家電、カメラ、ウェアラブル機器などの総称。

組み込みシステムに通信機能を組み合わせた機器も多い。

教材に記載された脆弱性

脆弱性	概要
通信・DoSのリスク	無線通信、複数の通信経路、再接続や再送による遅延・停止
デバイスのセキュリティ	盗難、物理攻撃、ファームウェア解析、データ抽出
暗号化の制約	CPU、メモリ、電力の制約により十分な暗号処理が難しい場合がある

また、センサーやカメラの周囲の環境を操作して、誤った測定結果を生じさせる攻撃もある。

低減策（CISSP・実務上の補足）

※教材には独立した低減策の記載なし。

- ネットワーク分離
- 初期パスワードの変更
- 強固な認証
- 機器の能力に応じた適切な暗号化
- 安全なファームウェア更新
- 物理的な保護
- 資産管理と監視

CISSP重要ポイント

IoTでは、機器数の増加、物理的な露出、リソース制約が主要なリスクとなる。

組み込みシステムとの違い：

- 組み込みシステム：特定機能を実行する機器内部のコンピューター。
- IoT：ネットワーク接続を通じてデータ交換や遠隔制御を行う機器・仕組み。

両者は重複する概念であり、必ずしも排他的ではない。

8. 分散システム（Distributed Systems）

概要

複数の独立したコンピューター（ノード）が、ネットワークを通じて処理やストレージを分担するシステム。

例：

- クラウド環境
- 分散データベース
- データウェアハウス
- P2Pシステム

主なメリット：

- 負荷分散
- 拡張性
- 冗長性
- 耐障害性

主な脆弱性と低減策

教材の脆弱性	概要	主な低減策
監視機能の不足	ノードの障害や侵害を迅速に検知できない	システム監視、IDS
アクセス制御の不足	異なる機密区分の情報を適切に保護できない	アクセス制御の強化
追加機能の制約	既存構成では十分な保護機能を追加できない	物理的・論理的・管理的な分離

CISSP重要ポイント

分散化は可用性を高められるが、監視やアクセス制御を複雑にする。

- レプリケーション：データを複数の場所に複製すること。
- 動的負荷分散：負荷に応じて処理を自動的に振り分けること。
- Graceful Degradation：一部の構成要素が故障しても、性能を段階的に低下させながら機能を維持する設計。

分散システムだからといって、必ず耐障害性が高いわけではない。

覚え方：分散で止まりにくくなるが、守る対象も増える。

9. 仮想化システム（Virtualized Systems）

概要

物理的なコンピューティングリソースを抽象化し、複数の仮想環境を構築する仕組み。

主なメリット：

- ハードウェア利用効率の向上
- コスト削減
- 環境の迅速な構築
- 柔軟なリソース配分
- クラウド環境の基盤技術

主な脆弱性と低減策

教材の脆弱性	概要	主な低減策
VMスプロール	管理されていないVMが増殖する	VMのライフサイクル管理、変更管理
VMエスケープ		

CISSP Domain 3：15種類のシステムアーキテクチャー比較マトリクス

No.	分類	アーキテクチャー	一言でいうと	具体例・製品	主な脆弱性・リスク	主な低減策	覚え方・CISSPのポイント
1	基本システム	クライアントベース	利用者が操作する端末	Windows PC、Mac、スマートフォン、POS端末	マルウェア、紛失・盗難、端末管理不足	EDR、パッチ管理、MDM、暗号化	**クライアント＝利用者の入口。**エンドポイントを継続的に保護する
2	基本システム	サーバーベース	サービスを提供する基盤	Windows Server、Linux Server、Apache、Nginx	サービス停止、更新困難、設定不備	冗長化、パッチ管理、最小権限、監視	**サーバー＝サービス提供者。**可用性とセキュリティを両立する

No.	分類	アーキテクチャー	一言でいうと	具体例・製品	主な脆弱性・リスク	主な低減策	覚え方・CISSPのポイント
3	基本システム	暗号システム	データを暗号技術で保護する	AES、RSA、ECC、TLS、HSM	鍵漏えい、弱い暗号、実装不備	標準暗号の採用、鍵管理、安全な実装	**暗号＝データを守る。鍵管理が生命線。**独自暗号を避ける
4	基本システム	データベース	データを保存・検索・管理する	MySQL、PostgreSQL、Oracle Database、SQL Server	SQLインジェクション、DoS、権限回避	入力検証、パラメータ化クエリ、最小権限、監査	**DB＝情報の保管庫。**機密性・完全性・可用性を守る
5	物理・デバイス系	ICS	工場や社会インフラを制御する	SCADA、DCS、PLC、工場の制御システム	設備停止、物理的被害、更新困難、長い製品寿命	ネットワーク分離、変更管理、監視、アクセス制御	**ICS＝工場・インフラ。**安全性と可用性を特に重視する
6	物理・デバイス系	組み込みシステム	機器の内部で特定機能を実行する	自動車のECU、家電の制御基板、医療機器の制御装置	実装不備、弱い認証、更新困難、リバースエンジニアリング	セキュア設計、ファームウェア更新、認証強化、代替的保護策	**組み込み＝機器の頭脳。**性能・電力・更新の制約が大きい
7	物理・デバイス系	IoT	ネットワークにつながる機器	スマート家電、ネットワークカメラ、温度センサー、スマートウォッチ	通信攻撃、物理攻撃、認証不備、暗号化の制約	機器管理、認証、通信保護、ネットワーク分離	**IoT＝つながる機器。**接続によって攻撃対象領域が広がる
8	分散・基盤系	分散システム	複数の独立したコンピューターが協調動作する	複数台のWebサーバー、別マシン上のWeb・AP・DB、Hadoop	全体監視の不足、アクセス制御の複雑化、通信障害	統合監視、アクセス制御、ノード間通信の保護	**分散＝複数ノードの協調。**冗長化は目的の一つであり必須条件ではない
9	分散・基盤系	仮想化	物理リソースを抽象化・分割する	VMware ESXi、Hyper-V、KVM、仮想マシン	VMスプロール、VMエスケープ、ハイパーバイザー侵害	変更管理、IAM、仮想基盤の保護、構成管理	**仮想化＝物理資源を分ける。**VMの隔離は絶対ではない

No.	分類	アーキテクチャー	一言でいうと	具体例・製品	主な脆弱性・リスク	主な低減策	覚え方・CISSPのポイント
10	分散・基盤系	クラウド	ITリソースをネットワーク経由で必要に応じて利用する	AWS、Microsoft Azure、Google Cloud	設定ミス、更新不足、責任分担の誤認	責任共有モデル、構成管理、契約管理、IAM	**クラウド＝リソースをサービスとして利用。** CSPと利用者の責任分担が重要
11	アプリケーション実行系	マイクロサービス	アプリケーションを小さな業務機能に分割する設計	ECサイトの注文・決済・在庫管理サービス、Spring Bootで構築した独立サービス	過剰権限、サービス間通信の攻撃、DoS、管理の複雑化	サービス間認証・認可、最小権限、APIゲートウェイ	**マイクロサービス＝業務機能で分ける設計。** サービス間の認証・認可が重要
12	アプリケーション実行系	コンテナ	アプリケーションを隔離された実行環境で動かす	Docker、containerd、Kubernetes、Nginxコンテナ、MySQLコンテナ	危険なイメージ、過剰権限、ホストとの隔離不足	イメージ管理、不変インフラ、最小権限、通信制限	**コンテナ＝アプリを動かす入れ物。** イメージとホストを保護する
13	アプリケーション実行系	サーバーレス	サーバー管理を意識せずコードやサービスを実行する	AWS Lambda、Azure Functions、Google Cloud Run functions	インジェクション、設定ミス、データ残留、過剰権限	セキュアコーディング、テスト、IAM、設定管理	**サーバーレス＝実行基盤の管理を委ねる。** コードの責任は利用者に残る
14	特殊用途・配置系	HPC	多数の計算資源を使って高速に並列計算する	スーパーコンピューター「富岳」、気象シミュレーション、科学技術計算	セキュリティ機能による遅延、不正ワークロード	境界保護、監視、ログ管理、利用者管理	**HPC＝超高速な並列計算。** 性能とセキュリティの両立が課題
15	特殊用途・配置系	エッジ・フォグ	データ発生源の近くに処理を配置する	工場内のエッジ端末、IoTゲートウェイ、ローカル分析サーバー	ネットワーク侵害、機器の物理的侵害、攻撃対象領域の拡大	ネットワーク監視、資産管理、アクセス制御、インシデント対応	**エッジ＝現場、フォグ＝現場周辺、クラウド＝遠隔。** 分散機器の管理が重要

5つのグループによる暗記

グループ	対象	共通する特徴	覚え方

グループ	対象	共通する特徴	覚え方
基本システム	クライアント、サーバー、暗号、データベース	情報の利用・提供・保護・保存	**使う・提供する・守る・保存する**
物理・デバイス系	ICS、組み込み、IoT	現実世界の機器と関係する	**工場・機器の頭脳・つながる機器**
分散・基盤系	分散システム、仮想化、クラウド	計算資源の配置・分割・提供	**連携・分割・提供**
アプリケーション実行系	マイクロサービス、コンテナ、サーバーレス	アプリケーションの設計・実行	**設計・入れ物・実行基盤の委託**
特殊用途・配置系	HPC、エッジ・フォグ	計算性能や処理場所の最適化	**高速計算・現場処理**

今回のディスカッションで混同しやすかった概念

比較対象	違い	具体例	覚え方
3層構成 vs 分散システム	3層構成は機能・役割の分離。分散システムは複数ノードの協調動作	Web・AP・DBを別マシンに配置すれば、3層構成かつ分散システムの一例	**3層＝役割、分散＝ノード**
分散システム vs 冗長化	分散システムは複数ノードの協調。冗長化は故障時に備えて機能を重複させること	Webサーバー3台の負荷分散構成は分散システムであり、冗長化にも利用できる	**分散＝構成、冗長化＝耐障害性を高める手段**
仮想化 vs コンテナ	VMはハイパーバイザーを使って仮想マシンを動かす。コンテナは通常ホストOSのカーネルを共有する	VMware ESXiとDocker	**VM＝仮想マシン、コンテナ＝軽量な実行環境**
マイクロサービス vs コンテナ	マイクロサービスはアプリケーションの設計。コンテナはアプリケーションの実行環境	注文サービスをDockerコンテナで動かす	**マイクロサービス＝何を分けるか、コンテナ＝どう動かすか**
コンテナ vs サーバーレス	コンテナは実行環境の技術。サーバーレスは実行基盤の管理を事業者に委ねるモデル	自前で管理するDocker環境とAWS Lambda	**コンテナ＝入れ物、サーバーレス＝基盤管理の委託**
エッジ vs フォグ	エッジはデータ発生源の近くで処理。フォグはエッジとクラウドの間の複数階層にも処理を分散	センサー直近の処理装置と工場内の集約・分析サーバー	**エッジ＝現場、フォグ＝現場周辺**
エッジ・フォグ vs オンプレミス	エッジ・フォグは処理場所の考え方。オンプレミスは設備の設置・運用形態	工場内に設置したエッジ機器は、オンプレミスでもあり得る	**エッジ＝データとの距離、オンプレ＝設置・運用形態**

CISSP試験で押さえる共通原則

1. **資産管理**：何が存在し、誰が管理しているかを把握する。
2. **リスク評価**：脆弱性だけでなく、脅威と影響も考慮する。
3. **セキュアな設計**：最小権限、分離、多層防御を適用する。
4. **変更・更新管理**：可用性や安全性への影響を考慮して更新する。
5. **継続的な監視**：異常を検知し、適切に対応する。
6. **責任の明確化**：システム所有者、運用者、クラウド事業者などの責任範囲を整理する。
7. **継続的改善**：評価・監視の結果を設計や運用に反映する。

最重要の暗記フレーズ

- **ICSは安全性と可用性**
- **組み込みは機器の制約**
- **IoTは接続による攻撃面の拡大**
- **分散は複数ノードの協調**
- **仮想化は物理資源の分割**
- **クラウドはサービスとしての提供と責任共有**
- **マイクロサービスは業務機能の分割**
- **コンテナは実行環境の隔離**
- **サーバーレスは実行基盤の管理委託**
- **HPCは性能とセキュリティの両立**
- **エッジは現場、フォグは現場周辺**

CISSP Domain 3｜3.6 暗号ソリューションを選択・決定する

1. この章の目的と全体像

組織の情報セキュリティ要件に合わせ、適切な暗号方式、デジタル証明書、鍵管理・運用方法を選択する。**暗号の数学的証明より「何を守るために、どの技術を選び、どう運用するか」が重要。**

守りたいもの	代表的な手段	注意点
機密性	対称暗号、公開鍵暗号、TLS	鍵が漏れれば保護できない
完全性	ハッシュ、MAC、デジタル署名	単純なハッシュ値の比較だけでは送信者を認証できない
真正性	MAC、デジタル署名、証明書	誰が作成・送信したかを確認
可用性	冗長化、復旧手順、適切な暗号運用	暗号化だけで可用性は保証されない
否認防止	デジタル署名、時刻証明、監査証跡	受信の証明には受領確認等も必要
アクセス制御	認証、鍵管理、暗号化による利用制限	暗号はアクセス制御を支援するが、それ自体が全てではない

CIAとParkerian Hexad：CIAは機密性・完全性・可用性。Parkerian Hexadはこれに真正性（Authenticity）、占有・管理（Possession/Control）、有用性（Utility）を加えた6要素。否認防止やアクセス制御は重要なセキュリティサービスだが、Hexadの追加3要素そのものではない。

2. なぜ暗号技術なのか

暗号は古代から情報の内容を第三者に理解させないために利用されてきた。現代は秘匿だけでなく、完全性・真正性・署名・鍵交換にも利用される。

用語	意味
平文（Plaintext）	暗号化前のデータ。文章、画像、音声、動画、プログラム、DBレコードなど全てが対象
暗号文（Ciphertext）	暗号化後のデータ
暗号化（Encryption）	平文を鍵とアルゴリズムで暗号文へ変換
復号（Decryption）	暗号文を平文に戻す
暗号（Cipher）	暗号化・復号の変換方式
ワークファクター（Work Factor）	攻撃者が解読に要する計算量・時間・費用などの労力
暗号システム（Cryptosystem）	アルゴリズム、鍵、機器、ソフトウェア、運用手順などを含む全体
鍵管理（Key Management）	鍵の生成、配布、保管、使用、更新、失効、廃棄

3. 暗号の進化（歴史）

時代	主な技術・人物	覚えるポイント
初期・手動時代	シーザー暗号、単純置換、転置、秘密の書記法	文字を置き換える／並べ替える
中世〜近世	アルベルティ、多表式暗号、ヴィジュネル暗号	単一の置換表から複数の置換表へ発展
機械の時代	文字を数値として扱う、暗号円盤・暗号機、鍵の工夫	A=0、B=1、…、Z=25のような数値化と機械処理
電気機械の時代	エニグマ、ローター式暗号機	鍵設定・運用手順・鍵のローテーションが重要
現代	DES、AES、RSA、ECC、TLS、商用暗号サービス	大量データは対称暗号、鍵共有・署名等は公開鍵技術

歴史の補足：アルベルティは15世紀の多表式暗号の先駆者。ヴィジュネル暗号の名称と発明史は単純ではなく、現在のヴィジュネル方式に近い記述はベラソ（1553年）にも見られる。CISSPでは細かな人名・年号より方式の進化を優先。

3.1 置換・転置・頻度分析

用語	内容	例・覚え方
置換暗号（Substitution）	文字や記号を別の文字・記号に置き換える。位置は維持	A→D、B→E

用語	内容	例・覚え方
転置暗号（Transposition）	文字そのものを変えず並び順を変える	ABCD→CADB
シーザー暗号	一定数だけアルファベットをずらす単純置換	3文字ずらす
ROT13	13文字ずらすシーザー暗号	2回適用すると元に戻る。**セキュリティ用途には不適**
デコードリング／暗号円盤	文字の対応を機械的に変える補助器具	置換表を回す
単表式暗号（Monoalphabetic）	基本的に一つの固定された置換対応を使う	同じ平文文字→同じ暗号文文字
多表式暗号（Polyalphabetic）	位置や鍵に応じて複数の置換対応を使う	ヴィジュネル暗号
頻度分析（Frequency Analysis）	暗号文の文字の出現頻度から平文を推測	英文でEが頻出する性質を利用
秘密の書記法（Secret Writing）	内容や存在を隠す書記・伝達方法の総称	暗号とステガノグラフィの歴史的文脈

頻度分析の要点：単表式置換は同じ文字が同じ文字へ変わるため頻度の偏りが残る。多表式暗号は単純な頻度分析に強くなるが、反復鍵などの弱点を攻撃され得る。

3.2 ランニング鍵暗号・ヴィジュネル暗号とmod 26

文字を A=0、B=1、…、Z=25 として扱い、鍵の文字に対応する数を加算する。

`暗号文 C = (平文 P + 鍵 K) mod 26`

例：平文 `C`（2）と鍵 `D`（3）なら、`(2+3) mod 26 = 5` → 暗号文 `F`。

- **mod 26**：26で割った余り。25を超えたら0に戻る（アルファベットを一周する）。
- **ヴィジュネル暗号**：短い鍵を繰り返し使用する代表的な多表式暗号。
- **ランニング鍵暗号（Running-Key Cipher）**：文章などの長い文字列を鍵として利用する方式。鍵が自然言語だと統計的偏りが残り得る。
- **ワンタイムパッド（OTP）**：真にランダムで平文と同じ長さの鍵を**一度だけ**使い、秘密に保つと情報理論的に完全秘匿を実現。鍵配布・保管・再利用防止が難しい。
- **セッション鍵**：通信セッションごとに用いる鍵。**ワンタイムパッドと同義ではない。**

3.3 エニグマから現代暗号へ

エニグマはローターを使って文字の置換を変化させる電気機械式暗号機。強度は装置だけでなく鍵設定・運用の適切さにも依存する。現代では公開レビューされた暗号方式が広く使われる。**暗号方式が強くても鍵の漏えい、設定ミス、実装不備で破られる**ことがある。

4. 鍵の強度・検索空間・ケルクホフスの原理

4.1 鍵空間

nビットの鍵は理論上 2^n 通り（ただし全てのビット列が有効な鍵である場合）。

鍵長	理論上の組み合わせ
16ビット	65,536通り（0から65,535）
56ビット	約7.2×10^16通り
128ビット	約3.4×10^38通り
256ビット	約1.16×10^77通り（78桁）

用語	説明
鍵空間（Key Space）	有効な鍵の候補の集合
検索空間（Search Space）	攻撃者が探索する候補全体
ブルートフォース攻撃	鍵やパスワード候補を総当たりする
ボットネット	侵害した多数の端末を使う攻撃基盤。計算分散に悪用され得る
レインボーテーブル	事前計算を用いてパスワードハッシュ等の照合を効率化する手法。ソルトで使い回しを抑止
弱い鍵・鍵空間の偏り	鍵生成が偏ると、見かけの鍵長ほどの強度が得られない
決定論的復号	同じ正当な鍵・暗号文に対して所定の平文を復元する性質

注意：ハッシュの衝突耐性・原像計算困難性はハッシュ関数の要件であり、全ての暗号化方式に同じ意味で適用するものではない。暗号の安全性は鍵長だけでなく、アルゴリズム・モード・乱数・実装・運用に依存する。

4.2 ケルクホフスの原理

「暗号方式が公開されても、鍵が秘密なら安全であるべき」という原則。

- 秘密にするのはアルゴリズムではなく鍵。
- 独自の未検証暗号より、公開され十分に検証された方式を選ぶ。
- Security through Obscurity（仕組みの隠蔽）だけに依存しない。

5. 暗号システムアーキテクチャ

5.1 ストリーム暗号とブロック暗号

比較	ストリーム暗号	ブロック暗号
処理単位	ビット／バイトなどの連続データ	固定長ブロック（AESは128ビット）
仕組み	鍵ストリームと平文をXORするなど	ブロックごとに変換。モードを使って長いデータを処理
例	ChaCha20、旧RC4	AES、旧DES、Twofish

比較	ストリーム暗号	ブロック暗号
注意	同じ鍵ストリームの再利用は危険	ECBはパターンが漏れる。モード・IV/nonceの扱いが重要

XOR（排他的論理和）：入力ビットが異なると1、同じなら0。ストリーム暗号などで利用。

CBC・CTR・GCMはブロック暗号の利用モードの例。CBC-MACは認証用のMAC構成であり、CBC暗号化モードそのものとは区別する。

5.2 対称鍵と非対称鍵

比較	対称鍵暗号	非対称鍵暗号
鍵	同じ共有秘密鍵	公開鍵と秘密鍵のペア
速度	高速。大量データ向き	相対的に低速。鍵交換・署名などに利用
主な課題	安全な鍵配布	公開鍵の真正性、計算負荷、鍵管理
例	AES、ChaCha20	RSA、ECCベースの暗号・署名、ElGamal

トラップドア（一方向）関数：一方向の計算は容易だが逆方向は困難で、特別な秘密情報があると逆方向を計算できる仕組み。公開鍵暗号の理解に重要。

5.3 ハイブリッド暗号

1. 公開鍵技術で安全に共有秘密を確立する、またはセッション鍵を暗号化して渡す。
2. 大量の実データは高速な対称鍵暗号で暗号化する。
3. 必要に応じて証明書・署名・認証付き暗号を組み合わせる。

代表例：TLS。現代のTLS 1.3は通常、（EC）DHEによる鍵共有、証明書等による認証、AES-GCMやChaCha20-Poly1305などの認証付き暗号を組み合わせる。SSLや古いTLSは安全性上の理由で利用を避ける。

6. 対称暗号化アルゴリズム

6.1 DES・AESと公開コンペティション

方式	概要	現在の位置づけ
DES	IBMの設計を基に標準化。56ビット鍵・64ビットブロック	鍵長不足で非推奨
3DES（TDEA）	DESを複数回適用	新規の暗号化用途では利用終了・移行対象
AES	NISTの公開選定を経てRijndaelを採用。128ビットブロック、128/192/256ビット鍵	現代の標準的な対称ブロック暗号

公開コンペティションや第三者の暗号解析は、アルゴリズムの透明性と信頼性向上に貢献する。

6.2 その他の代表的な暗号

方式	種類・特徴	覚え方
RC6	AES候補だったブロック暗号	AES最終候補
Twofish	AES最終候補のブロック暗号	Blowfishの後継的設計
Blowfish	Bruce Schneierが設計した64ビットブロック暗号、Feistel構造	古い64ビットブロック
RC5	可変のパラメータを持つブロック暗号	可変設定
RC4	ストリーム暗号	WEP・古いTLSで使用。現在は危険・非推奨
IDEA	64ビットブロックの対称暗号	歴史的にPGP等で利用
CAST	Carlisle AdamsとStafford Tavaresに由来する暗号ファミリー	CAST-128など
SAFER	Secure And Fast Encryption Routine系の暗号	Bluetoothで使われたSAFER+など

Feistel（ファイステル）構造：ブロックを左右に分け、ラウンド関数とXORを繰り返す構造。DES、Blowfishなどで利用。AESはFeistel構造ではない。

6.3 対称暗号の利点・欠点

- **利点**：高速・大量データ向き・比較的低負荷。
- **欠点**：共有秘密鍵の配布・保管・更新が難しい。共有鍵だけでは原則として否認防止を提供しない。
- **インバンド鍵配送**：データ通信と同じ経路で鍵情報をやり取りする（保護が必要）。
- **アウトオブバンド鍵配送**：別の経路で鍵を渡す（例：対面・別の安全なチャネル）。別経路だから自動的に安全とは限らない。

7. 非対称暗号・公開鍵と秘密鍵

7.1 数学的な基盤と代表方式

分類	代表方式	何が難しいか
整数の素因数分解	RSA	大きな合成数を素因数へ分解すること
有限体の離散対数	Diffie–Hellman、ElGamal	`g^x mod p`からxを求めること
楕円曲線の離散対数	ECDH、ECDSAなどのECC	楕円曲線上の点の倍算から元の倍率を求めること

補足：ECCは素因数分解ではない。`N = P × Q`からPとQを求めるのが難しいという例はRSAの基盤。ECCは楕円曲線上の離散対数問題を利用する。

7.2 RSA・ElGamal・Diffie–Hellman・ECC

方式	主な役割	注意
RSA	公開鍵暗号化、デジタル署名	安全なパディング（OAEP、PSS等）と適切な鍵長が必要

方式	主な役割	注意
ElGamal	公開鍵暗号化、署名方式の系譜	離散対数問題を利用
DH / ECDH	鍵共有・鍵合意	**単独では相手の認証を提供しない**。証明書・署名等と組み合わせる
ECC	楕円曲線を使う公開鍵技術の総称	小さい鍵長で高い安全性を実現しやすい

暗号スイート（Cipher Suite）：鍵交換・認証・暗号化・完全性保護などの技術の組み合わせ。RSA、DH、ECCは同じ機能の単一アルゴリズムとして一括りにしない。

7.3 ECCのイメージと鍵長比較

楕円曲線暗号では、曲線上の基準点Gを秘密の整数d回「加算」して公開点Qを得る。

公開鍵 `Q = d × G`（楕円曲線上の点演算）

- 秘密鍵dとGからQを計算するのは容易。
- GとQからdを逆算するのは困難（楕円曲線離散対数問題）。
- **グラフの形自体を暗記する必要はない**。点演算が容易で逆算が難しいという性質が重要。

おおよそのセキュリティ強度	RSA鍵長	ECC鍵長
128ビット相当	3,072ビット	256ビット程度
192ビット相当	7,680ビット	384ビット程度

※鍵長の比較は同等のセキュリティ強度の目安であり、アルゴリズムやパラメータ選定が前提。

7.4 公開鍵による暗号化とデジタル署名

目的	送信者が使う鍵	受信者が使う鍵	何を得るか
機密性	**受信者の公開鍵**で暗号化	**受信者の秘密鍵**で復号	受信者以外が読みにくい
署名・発信元の証明	**送信者の秘密鍵**で署名	**送信者の公開鍵**で検証	完全性・署名者の真正性・否認防止の支援

注意：公開鍵は一般に秘密鍵と対になる形で生成され、CAが通常公開鍵そのものを生成するわけではない。 CAは公開鍵と主体の結び付きを証明書として署名する。

7.5 秘密メッセージ＋発信元証明（現代的な流れ）

1. 送信者がメッセージのハッシュ値を計算する。
2. 送信者の**秘密鍵でデジタル署名**を作成する。
3. メッセージと署名を、受信者との間で確立した対称鍵で暗号化する（その鍵は公開鍵技術等で安全に共有）。
4. 受信者が対称鍵で復号する。
5. 受信者が送信者の公開鍵で署名を検証する。

教材にある「送信者の秘密鍵でメッセージ本文を暗号化→受信者の公開鍵でさらに暗号化」という説明は、署名と暗号化を理解するための古典的な簡略モデル。 実際のデジタル署名は通常「本文全体を秘密鍵で暗号化」することではない。また、公開鍵暗号で暗号化できるデータ長には方式ごとの制限があり、「64KB」という値は一般的な一律の上限ではない。大量データにはハイブリッド暗号を用いる。

7.6 非対称暗号の利点・欠点

- **利点**：事前の共有秘密なしに鍵共有を支援し、デジタル署名による真正性・否認防止の支援が可能。
- **欠点**：対称暗号より計算負荷が高く、大量データの直接暗号化には不向き。
- **原則**：公開鍵技術で鍵共有・署名、対称鍵で本文暗号化。

8. 量子暗号とポスト量子暗号

用語	概要
量子鍵配送（QKD）	量子状態を利用して鍵配送時の盗聴検知を目指す技術。専用の物理的設備などが必要
ポスト量子暗号（PQC）	量子計算機による攻撃に耐えることを目指す**古典コンピュータ上で実行できる**暗号方式
Open Quantum Safe	PQCアルゴリズムの試験・統合などを支援するオープンソースプロジェクト
Shorのアルゴリズム	大規模な誤り耐性量子計算機が実現すればRSAや従来のDH/ECCに重大な影響
Groverのアルゴリズム	対称鍵の総当たり探索に対する理論的な高速化。鍵長選定で考慮

8.1 NISTのPQC標準

標準	アルゴリズム	主な用途
FIPS 203	ML-KEM（旧CRYSTALS-Kyber）	鍵カプセル化・共有鍵確立
FIPS 204	ML-DSA（旧CRYSTALS-Dilithium）	デジタル署名
FIPS 205	SLH-DSA（SPHINCS+系）	ハッシュベースのデジタル署名

補足：Falconは別系統の署名方式として標準化が進められているが、上記3つのFIPSに含まれるものではない。**QKDとPQCは別物。**

9. 暗号の特殊な用途

用語	仕組み	CISSPのポイント
ステガノグラフィ	画像・音声などに情報を埋め込み、**情報の存在自体**を隠す	暗号化は内容を隠す。ステガノグラフィは存在を隠す
分散台帳（DLT）	複数の参加者で記録を共有・検証する	単一管理者への依存を減らす設計が可能
ブロックチェーン	ハッシュで前のブロックとつなぎ、改ざんを検出しやすくする分散台帳の一種	ハッシュ・署名・合意形成を組み合わせる。**記録の真実性まで自動保証しない**

用語	仕組み	CISSPのポイント
E2EE（エンドツーエンド暗号化）	送信端末で暗号化し、意図した受信端末で復号する	中継サーバーでも通常は本文を読めない設計
リンク暗号化	通信リンクごとに暗号化・復号する	中継装置では平文になる場合がある
バルク暗号化	大量のデータをまとめて効率的に暗号化する	対称鍵暗号が向く。**リンク暗号化と同義ではない**

10. 日常的に使われる暗号化：ハイブリッドシステムとPKI

10.1 ハイブリッド暗号によるメッセージ交換

例1：秘密メッセージ交換

1. Aliceがランダムな対称鍵（セッション鍵）を生成。
2. 本文を対称鍵で暗号化し、暗号文メッセージ（CTM）を作成。
3. 対称鍵をBobの公開鍵で保護し、暗号化された鍵（CTK）を作成（方式により鍵カプセル化等）。
4. Bobは秘密鍵でセッション鍵を回復し、本文を復号する。

例2：秘密性＋発信元証明

上記に加え、Aliceがメッセージダイジェスト（ハッシュ）に対してデジタル署名を付ける。BobはAliceの証明書・公開鍵で署名を検証する。**送信証明と受信証明は別**であり、受信の否認防止には署名付き受領確認などが必要。

10.2 PKI（Public Key Infrastructure：公開鍵基盤）

PKIは公開鍵を本人・組織・サーバー等の主体と結び付け、証明書の発行・検証・失効を管理する仕組み。

役割	正式名称	主な仕事
CA	Certification Authority（認証局）	証明書を発行・電子署名する
Root CA	ルート認証局	信頼の起点。自己署名証明書を持つ
Intermediate CA	中間認証局	上位CAから認証され、下位証明書を発行
RA	Registration Authority（登録局）	証明書申請者の本人確認・登録を担当
VA	Validation Authority（検証局）	証明書の有効性・失効状態の確認を支援

`Root CA → Intermediate CA → サーバー等の証明書`

信頼の連鎖（Chain of Trust）：末端証明書から中間CA、信頼済みルートCAまで署名を検証する。CAが侵害されると不正な証明書が発行され得る。2011年のDigiNotar事件は、CAの信頼失墜の代表例。

10.3 X.509デジタル証明書

X.509は公開鍵証明書の標準形式。主体の公開鍵と識別情報をCAの署名で結び付ける。

フィールド	意味

フィールド	意味
バージョン	X.509証明書の形式バージョン（一般にv3）
シリアル番号	CAが発行する証明書の識別番号
署名アルゴリズム	CAが証明書へ署名する際の方式
発行者（Issuer）	証明書を発行したCA
有効期間（Not Before / Not After）	証明書が有効な開始・終了日時
主体（Subject）	証明書の対象となる主体
主体の公開鍵情報	公開鍵とそのアルゴリズム
発行者固有ID／主体固有ID	任意の識別フィールド（現在は一般的でない）
拡張（Extensions）	SAN、Key Usage、Basic Constraintsなどの追加情報
CAのデジタル署名	証明書内容がCAによって署名されたことを検証する

注意：証明書に秘密鍵は格納しない。HTTPSではSAN（Subject Alternative Name）による接続先名の検証が重要。

10.4 証明書の失効：CRLとOCSP

方式	仕組み	特徴
CRL（Certificate Revocation List）	CAが失効した証明書の一覧を発行	一覧取得方式。更新間隔による遅延がある
OCSP（Online Certificate Status Protocol）	対象証明書の状態をオンラインで問い合わせ	個別照会方式。応答は`good`、`revoked`、`unknown`

OCSPのgoodは「失効していないと応答した」という意味で、証明書の全ての条件を保証するものではない。 期限、名前、署名チェーン、用途なども検証する。OCSP staplingではサーバーがOCSP応答を添付する。

11. ハッシュデータとメッセージ完全性

11.1 ハッシュ関数とは

任意長の入力から一定長のダイジェスト（ハッシュ値）を生成する。**ハッシュは暗号化と異なり、復号を前提としない。**

重要な性質	意味
一様分布	出力が偏りにくい
衝突耐性	異なる入力で同じハッシュ値を見つけにくい
原像計算困難性（一方向性）	ハッシュ値から元の入力を求めにくい
メッセージ全体への依存	入力の一部が変わると出力も大きく変化し得る（雪崩効果）

重要な性質	意味
決定論的	同じ入力なら同じハッシュ値を返す

※衝突は有限長出力のため数学的には必ず存在する。「衝突しない」ではなく「実用的に見つけにくい」が正確。

11.2 代表的なハッシュアルゴリズム

方式	出力長	状況・特徴
MD5	128ビット	衝突攻撃が実用化。セキュリティ用途の署名等には不適
SHA-1	160ビット	衝突攻撃が実証。新規のデジタル署名用途では使用しない
SHA-224	224ビット	SHA-2ファミリー
SHA-256	256ビット	SHA-2。広く利用
SHA-384	384ビット	SHA-2
SHA-512	512ビット	SHA-2
SHA-512/224・SHA-512/256	224・256ビット	SHA-512系の派生
SHA-3	224/256/384/512ビット等	Keccakベース。SHA-2とは別構造
RIPEMD-160	160ビット	歴史的な利用例があるハッシュ

重要な訂正：FIPS 180-4にはSHA-1も記載されるが、**現在SHA-1を新しいデジタル署名に使うことを推奨しているわけではない**。標準に記載があることと推奨されることは別。

11.3 メッセージ完全性コントロール（MIC）

教材の図の意図は「偶発的変更を検出する仕組み」と「悪意ある改ざんにも対抗する仕組み」の区別。

主な目的	技術	説明
偶発的な変更の検出	パリティ	ビット誤りの検出
偶発的な変更の検出	チェックサム、CRC	伝送・保存時の破損を検出
偶発的な変更の検出	通常のハッシュ	データが変わったか確認。ただしハッシュ値自体の真正性は別途必要
意図的な変更への対抗	鍵付きハッシュ／HMAC	共有秘密鍵を用いた認証コード
意図的な変更への対抗	CBC-MAC	ブロック暗号を用いるMAC構成

主な目的	技術	説明
意図的な変更への対抗	デジタル署名	秘密鍵で署名し公開鍵で検証

重要：ハッシュ単体は攻撃者が本文とハッシュ値を両方置き換えられる場合、改ざんを防げない。

11.4 MAC・HMAC・CBC-MAC

技術	仕組み	提供する性質
MAC（Message Authentication Code）	共有秘密鍵とメッセージから認証コードを生成	完全性＋共有鍵に基づく真正性
HMAC	ハッシュ関数と共有秘密鍵を組み合わせるMAC	同上
CBC-MAC	ブロック暗号のCBC型処理を使うMAC	同上。可変長メッセージでは構成上の注意が必要

MACは一般に元のメッセージより短い。MACから元のメッセージを復元することや、鍵を知らずに有効な別のメッセージ・MACの組を偽造することは困難であるべき。

MAC/HMACは共有鍵方式なので、原則として否認防止を提供しない。

12. 否認防止（Nonrepudiation）

送信者や受信者が、後から自分の行為を否定しにくくする仕組み。

対象	何を証明するか	代表的な手段
送信の否認防止	送信者がメッセージを作成・送信したこと	デジタル署名、監査証跡、時刻証明
完全性	メッセージが変更されていないこと	署名・ハッシュの検証
受信の否認防止	受信者が受け取ったこと	署名付き受領確認、配達記録等

注意：送信者の署名だけでは、相手が実際に受信したことまでは証明できない。

13. デジタル署名（Digital Signature）

13.1 基本の仕組み

1. 送信者がメッセージのハッシュを計算する。
2. **送信者の秘密鍵**で署名する。
3. 受信者が**送信者の公開鍵**で署名を検証する。
4. 証明書・鍵の有効性も確認する。

提供するもの：完全性、署名者の真正性、否認防止の支援。**提供しないもの**：署名だけでの機密性。

NISTのデジタル署名標準DSSはFIPS 186シリーズ。教材のFIPS 186-4は歴史的な版であり、現在は**FIPS 186-5**が後継。

13.2 利用例

用途	何を確認するか
ソフトウェア更新・コード署名	提供元と改ざん有無
デバイスドライバー	署名者とファイルの完全性。**OS互換性や安全性を署名だけで保証しない**
電子契約・スマートコントラクト	署名者・合意内容・トランザクションの証跡
身分証・許認可・公的文書・電子投票	認証・記録保持・改ざん検知などを支援（制度と実装に依存）
公証・アポスティーユ・公印確認	文書の公的な認証手続を支援（デジタル署名と法的手続は別概念）
デジタルフォレンジック	証拠の完全性とChain of Custody（証拠保全の連鎖）を支援

14. 暗号のピースを組み合わせる

実際のセキュリティは単一技術ではなく、**暗号化＋ハッシュ＋署名＋PKI＋鍵管理＋運用**の組み合わせで実現する。

目的	組み合わせ例
安全なWeb通信	TLS、証明書、鍵共有、認証付き暗号
発信元と完全性の確認	ハッシュ＋デジタル署名＋PKI
秘密メッセージ交換	公開鍵技術による鍵確立＋対称暗号
否認防止の強化	デジタル署名＋本人確認＋時刻証明＋監査証跡

15. 暗号システムの衛生管理・運用・保守

サイバーハイジーン（Cyber Hygiene）：日常的に継続する基本的なセキュリティ習慣・運用。CIS ControlsのImplementation Group 1（IG1）は、基本的なサイバー衛生を実践するための優先的な対策群。

15.1 有用な衛生管理の6つの視点

1. 個人と組織の行動の両方に焦点を当てる。
2. 実行しやすく、組織規模に応じて展開しやすい具体的な行動にする。
3. 防止・検知・対応したい攻撃の種類を明確にする。
4. 過度に専門的な知識や複雑な管理を必要としない。
5. 導入と**測定**を支えるツール・技術が利用できる。
6. より包括的なセキュリティ改善につながる道筋を示す。

15.2 暗号管理（Cryptographic Management）の実務

管理項目	主な対策
暗号方式の選定	標準化された強い方式を採用。MD5・SHA-1・RC4・DES等の旧方式から移行
鍵生成	安全な乱数生成器と適切な鍵長

管理項目	主な対策
鍵の保管	HSM・TPM・KMS等を適切に活用しアクセス制御
鍵の配布	認証された安全な経路・プロトコル
鍵のローテーション	用途・リスク・ポリシーに応じて更新
鍵の失効・廃棄	漏えい時の迅速な無効化、適切な削除
証明書管理	有効期限・信頼チェーン・失効状態の監視
設定管理	安全なTLS設定、暗号スイート、nonce/IV再利用防止
監視・監査	鍵利用履歴、権限変更、失効、例外の記録
人材・運用	手順書、権限分離、教育、定期レビュー

※この暗号管理の表は、読み上げられた範囲を踏まえた実務的な補足。教材の「暗号管理」見出し以降の本文は未読のため、教材の全項目を再現したものではない。

16. CISSP試験直前：暗記表

問われたら	まず思い出すこと
大量データを高速に暗号化	AES等の対称暗号
安全に鍵を共有したい	（EC）DHや鍵カプセル化＋認証
改ざんを検出したい	ハッシュ（値の信頼性にも注意）
改ざんと共有鍵保有者を検証	HMAC / MAC
発信元を証明・否認防止を支援	デジタル署名
公開鍵と主体を結び付ける	PKI / X.509証明書
証明書の失効確認	CRL / OCSP
文字の出現頻度で暗号解読	頻度分析
文字を置き換える／順番を変える	置換／転置
暗号方式を公開しても安全	ケルクホフスの原理
情報の存在自体を隠す	ステガノグラフィ
端末間で本文を秘匿	E2EE
中継リンクごとに暗号化	リンク暗号化
大量データの一括暗号化	バルク暗号化
量子計算機を想定した公開鍵方式	PQC（ML-KEM、ML-DSA、SLH-DSA）
暗号が破られる典型的な原因	鍵管理・実装・設定・運用の不備

17. 最後に：5つの重要原則

1. **暗号化とハッシュは別**：暗号化は復号できるが、ハッシュは原則復元しない。
2. **対称暗号と非対称暗号は役割分担**：大量データは対称暗号、鍵確立・署名は公開鍵技術。
3. **署名と暗号化は別**：署名は真正性・完全性、暗号化は機密性。
4. **アルゴリズムだけで安全にはならない**：鍵管理・実装・証明書・運用が重要。
5. **最適な方式は要件から選ぶ**：機密性、完全性、真正性、可用性、否認防止、性能、運用負荷を総合評価する。

補足：教材を読む際の訂正ポイント

- 16ビットの鍵は**65,536通り**（最大値が65,535）。
- ECCの数学的基盤は**楕円曲線離散対数**。素因数分解はRSA。
- CAは通常、公開鍵を**生成する機関ではなく証明する機関**。
- デジタル署名は単に「本文を秘密鍵で暗号化すること」ではない。
- 署名はソフトウェアの互換性・無害性を自動保証しない。
- SHA-1は歴史的標準に載っていても、現在の新規署名用途には不適。
- セッション鍵とワンタイムパッドは別物。
- QKDとPQCは別物。
- 受信の否認防止には、送信者の署名とは別の受領証跡が必要。

3.7 暗号解読攻撃の方法を理解する

目的・概要

- **目的**：一般的な暗号解読攻撃と、それに対する防御を理解する。
- **暗号解読（Cryptanalysis）**：暗号アルゴリズム・鍵・実装・運用を分析し、弱点や性質を調べること。防御技術の研究にも悪用にも利用される。
- 攻撃対象は**暗号の数学的性質**だけではない。**人間、鍵管理、乱数、プロトコル、実装、物理的漏えい**も対象になる。
- **ワークファクター（Work Factor）**：攻撃に必要な計算量・時間・資源。鍵長だけでなく実装の品質にも左右される。

1. 鍵発見のためのソーシャルエンジニアリング

項目	内容
攻撃方法	偽装、フィッシング、詐欺、脅迫、買収などで人間から認証情報・秘密鍵・運用情報を引き出す
狙う弱点	暗号自体ではなく**人間と運用**。APTでも利用される
低減策	セキュリティ意識向上（Awareness）、教育・訓練、フィッシング耐性のあるMFA、最小権限、鍵へのアクセス制限、異常監視

項目	内容
事例	**2011年RSA SecurID侵害**：標的型フィッシングのExcel添付がFlashのゼロデイ脆弱性を悪用。トークン関連情報の侵害が問題となり、交換・対策に多額の費用を要した

試験の要点：強力な暗号でも、鍵や認証情報を人間から奪われれば保護は破られる。

2. 暗号文ベースの攻撃と暗号解析

攻撃者の手元に**何があるか**で分類する。目標は平文の復元、鍵の推定、暗号の弱点の発見など。

攻撃	攻撃者が利用できるもの	手法・特徴	防御の考え方
暗号文のみ攻撃（Ciphertext-only）	暗号文のみ	統計、繰り返し、頻度分析などから推測。情報が少なく難しい	安全な現代暗号、適切なモードと乱数・IV
既知平文攻撃（Known-plaintext）	対応する平文と暗号文の組	対応関係から鍵・規則を推測	既知平文に耐える暗号方式を採用
選択平文攻撃（Chosen-plaintext, CPA）	攻撃者が選んだ平文とその暗号文	入力を操作し出力の差を観察	CPA安全性を考慮した方式、適切なnonce/IV
適応的選択平文攻撃	過去の結果に応じて次の平文を選択	繰り返し試して情報を増やす	安全な暗号設計と暗号化APIの制御
選択暗号文攻撃（Chosen-ciphertext, CCA）	攻撃者が選んだ暗号文と復号結果（復号オラクル）	復号応答・エラー差などから情報を得る	**認証付き暗号（AEAD）**、エラー情報の統一、適切なパディング検証
適応的選択暗号文攻撃	結果に応じて次の暗号文を選ぶ	復号オラクルを反復利用。限定的アクセスを「ランチタイム攻撃」と呼ぶ文脈もある	CCA耐性、復号APIの制限
線形暗号解読	多数の既知平文・暗号文ペア	ブロック暗号の入出力と鍵の関係を**線形近似**し、統計的偏りから鍵を推測	十分に解析された暗号、適切なラウンド数
差分暗号解読	通常は選択した平文のペア	平文の**差分**が暗号文の差分に与える影響を調べる	差分解析に耐える暗号設計

歴史的事例

- **エニグマ**：ポーランドの数学者マリアン・レイェフスキらが、暗号文・手順等を分析し、機械構造の再構築と解読に貢献。
- **ミッドウェー海戦の「AF」**：水不足という偽情報を利用して日本側の通信内容を確認し、AFがミッドウェー島を指すと裏付けた事例。**選択平文の発想に近い事例**として説明されるが、厳密には通常の暗号化オラクルを

使ったCPAとは異なる。

3. 実装とアルゴリズムへの攻撃（読み上げた19項目）

3-1. 全体比較表

#	攻撃・関連項目	狙い／仕組み	代表的な対策・注意点
1	**総当たり攻撃（Brute Force）**	鍵空間の候補を順に試す。ログインのパスワード推測にも使われる	十分な鍵長、強い乱数、ログイン試行制限、MFA、監視
2	**パス・ザ・ハッシュ（PtH）**	パスワードを知らずに**NTLMハッシュ**を認証に悪用し横展開	NTLM利用削減、Credential Guard、資格情報保護、特権分離、SMBの保護
3	**Kerberosの悪用**	チケットやKDC関連の弱点、盗難資格情報、リプレイなどを悪用	KDC保護、時刻同期、チケット寿命管理、監視、強いアカウント管理
4	**中間者攻撃（MITM）**	通信経路に介入し盗聴・改ざん・なりすまし。鍵交換やハイブリッド暗号も標的	相互認証、証明書検証、TLSの適切な設定、鍵交換の認証
5	**サイドチャネル攻撃**	電力、電磁波、熱、処理時間などの副次情報から秘密を推測	定時間実装、漏えい低減、物理保護、耐タンパー設計
6	**故障解析攻撃（Fault Analysis）**	意図的に異常を発生させ、正常時との差から鍵・処理を推測	異常検出、結果検証、冗長計算、耐故障設計
7	**タイミング攻撃**	処理時間の違いから鍵や分岐条件を推測するサイドチャネル攻撃	**定時間処理**、秘密依存の分岐・メモリアクセス抑制
8	**プローブ攻撃（Probing）**	暗号モジュールの配線・周辺回路・信号を観測	シールド、耐タンパー、筐体保護、検知機構
9	**リプレイ攻撃**	過去の正当な通信・認証メッセージを再送し不正に受理させる	**nonce、タイムスタンプ、シーケンス番号、期限、チャレンジレスポンス**
10	**代数攻撃（Algebraic Attack）**	暗号の数学的関係を方程式などで表し、鍵や内部状態を推定	十分に解析された暗号・適切なパラメータ
11	**レインボーテーブル**	事前計算したハッシュ候補を使ってパスワードを推測	**個別ソルト**、Argon2id/scrypt/bcryptなどの低速なパスワードハッシュ
12	**頻度分析**	文字・語の出現頻度から古典的置換暗号を解読	現代の安全な暗号を使用。単純置換を使わない
13	**誕生日攻撃**	誕生日のパラドックスを利用しハッシュの衝突を探す	十分なハッシュ出力長、衝突耐性のあるSHA-2/SHA-3等
14	**因数分解攻撃**	RSAの公開値に含まれる大きな整数を素因数分解し秘密鍵導出を狙う	十分なRSA鍵長、適切な実装、耐量子暗号への移行計画

#	攻撃・関連項目	狙い／仕組み	代表的な対策・注意点
15	**辞書攻撃**	よく使う単語・パスワードの候補を優先的に試す	長い固有パスワード、パスワードマネージャー、MFA、試行制限
16	**乱数生成器への攻撃**	PRNGの予測可能性から鍵・nonce・IVなどを推定	CSPRNG、十分なエントロピー、乱数生成器の検証
17	**フォルトインジェクション（Fault Injection）**	電源・クロック・電磁ノイズ等で意図的に故障を注入	耐タンパー、異常検出、結果検証、物理的防御
18	**一時ファイルの悪用**	平文・鍵・中間データが一時ファイルやスワップに残る	安全な一時領域、権限制御、暗号化、データ消去、秘密情報の最小保持
19	**ランサムウェア／身代金攻撃**	ファイルを不正暗号化して復旧と引き換えに金銭を要求。窃取と脅迫を伴う場合も	オフライン/イミュータブルバックアップ、EDR、最小権限、分離、復旧訓練

※「ランサムウェアと身代金攻撃」はユーザーの指示に従い**1項目に統合**。Kerberosはプロトコル自体が攻撃ではなく、ここでは**悪用される実装・運用上の弱点**として整理した。

3-2. 総当たり攻撃：鍵長だけでは安心できない

理論上、nビットの均等な鍵空間は **2^n通り**。ただし、実効的な探索空間と攻撃コストは次の事情で小さくなる場合がある。

要因	なぜ危険か
弱い疑似乱数生成器（PRNG）	生成される鍵が偏る・予測可能になると、理論上の鍵空間より探索候補が少なくなる
クラウド・GPU・ボットネット	大規模並列化で単位時間当たりの試行回数が増える
不適切なセッション鍵管理	鍵の盗難・再利用・漏えいにより、総当たり自体が不要になる場合がある
アルゴリズムや実装の弱点	数学的解析、サイドチャネル、設定ミスで探索を短縮できる
他の暗号解析との組み合わせ	既知平文・差分解析などで候補を絞り込める
ソーシャルエンジニアリング	鍵・認証情報を直接奪うことで暗号強度を迂回する

要因	なぜ危険か
量子計算	将来の大規模な耐故障量子計算機はRSA/ECCに深刻な影響を与え得る。対称鍵の総当たりに対するGroverのアルゴリズムの影響は概ね平方根の高速化で、RSA/ECCへのShorの影響とは異なる

注意：暗号鍵へのオフライン総当たりと、オンラインログインへのパスワード推測は区別する。後者にはアカウントロック・レート制限・MFAが有効。

3-3. PtH、Kerberos、MITMを区別する

- **PtH**：主にWindows/NTLMで、**パスワードそのものを知らなくても盗んだNTハッシュを認証に利用**する。SMB経由の横展開などが典型。
- **Kerberos**：MIT開発のチケットベースのネットワーク認証プロトコル。**KDC**（AS/TGS）を信頼の中心とし、SSOを実現する。時刻同期とチケットの有効期間が重要。KDCは高価値の攻撃対象・単一障害点になり得るため、保護・冗長化が必要。
- **TGT/TGSチケットの盗用**はPtHと区別し、一般に**Pass-the-Ticket（PtT）**などと呼ぶ。Kerberosチケットを「採取して再利用する」話は、単純にPtHと同一視しない。
- **MITM**：暗号化されていても**相手の認証が不十分**なら、攻撃者が通信相手を装って鍵交換に割り込む可能性がある。適切な証明書検証・認証付き鍵交換が重要。

3-4. サイドチャネル、故障解析、フォルトインジェクション

分類	観測／操作	例	区別
サイドチャネル	正常動作中の副次情報を**観測**	電力解析、電磁波解析、タイミング解析	通常は受動的
故障解析	故障した出力と正常出力を**比較**	誤った署名・復号結果から鍵を推測	分析の方法
フォルトインジェクション	故障を**意図的に発生**させる	電圧グリッチ、クロック異常、電磁パルス	故障を起こす手段
プローブ	物理回路・信号を直接測定	テストプローブによる内部信号観測	物理的アクセスが重要

タイミング攻撃対策：秘密に依存しない定時間アルゴリズムを使う。**パディングでメッセージ長を揃えるだけでは、処理時間の漏えい全般を防げない。**

3-5. ハッシュ攻撃の違い

- **レインボーテーブル**：パスワード候補とハッシュの関係を事前計算して探索を高速化する。**ソルト**は同一パスワードでも異なるハッシュを作り、事前計算の再利用を困難にする。
- **誕生日攻撃**：nビットの理想的なハッシュの衝突探索は概ね **2^(n/2)** 回規模。例：256ビットのハッシュなら衝突探索は概ね2^128回規模。
- **辞書攻撃**：人が選びやすい単語・パスワードを優先。**総当たり攻撃の一種／効率化**として理解できる。
- **頻度分析**：自然言語の統計を使う**古典暗号**の解読。現代の強い暗号への一般的な解読法ではない。

3-6. RSAと量子計算

- **RSA**：大きな合成数の素因数分解の難しさを安全性の基盤の一つとする。十分な鍵長・適切なパディング・鍵生成が重要。
- **Shorのアルゴリズム**：十分な能力の量子コンピューターがあれば整数因数分解や離散対数を効率的に解く理論的アルゴリズム。**通常のコンピューターで素因数分解が劇的に速くなるという意味ではない。**
- 対策は鍵長の延長だけではなく、**ポスト量子暗号（PQC）への移行計画**が必要。

4. CISSP試験直前チートシート

問題文のキーワード	最も関連する攻撃／概念
暗号文しか持っていない	暗号文のみ攻撃
平文と暗号文の対応ペアを持つ	既知平文攻撃
平文を選び暗号化結果を見る	選択平文攻撃
暗号文を選び復号結果を見る	選択暗号文攻撃
入力と出力の**線形近似**	線形暗号解読
入力の**差分**と出力の差分	差分暗号解読
鍵候補を全部試す	総当たり攻撃
辞書にある語からパスワード推測	辞書攻撃
NTLMハッシュを使って認証	Pass-the-Hash
Kerberosチケットを盗んで再利用	Pass-the-Ticket
通信の間に割り込む	MITM
過去の正当な通信を再送	リプレイ攻撃
消費電力・電磁波・処理時間	サイドチャネル攻撃
エラーを意図的に発生	フォルトインジェクション
正常結果と異常結果を比較	故障解析
事前計算済みハッシュ表	レインボーテーブル
ハッシュの衝突、2^(n/2)	誕生日攻撃
文字の出現頻度	頻度分析
RSAの大きな整数を分解	因数分解攻撃
乱数の予測、鍵・nonceの推測	乱数生成器への攻撃
一時ファイルに残った平文・鍵	一時ファイルの悪用
不正暗号化と身代金要求	ランサムウェア
従業員へのフィッシング・買収	ソーシャルエンジニアリング

5. 試験で迷いやすい点

1. **強いアルゴリズム ≠ 安全なシステム**。実装・鍵管理・乱数・認証・人間まで見る。
2. **PtHとPtTは別**。NTLMハッシュを悪用するか、Kerberosチケットを悪用するか。
3. **サイドチャネルとフォルトインジェクションは別**。観測か、故障を起こすか。
4. **暗号化だけではリプレイやMITMを防げない**。認証・鮮度検証が必要。
5. **誕生日攻撃はハッシュの衝突探索**。レインボーテーブルは主にパスワード推測。
6. **対称鍵の量子耐性とRSA/ECCの量子耐性は同じではない**。
7. **最も有効な対策は攻撃対象に合わせて選ぶ**。例：弱い乱数にはCSPRNG、PtHには資格情報保護、リプレイにはnonce。

3.8 サイトと施設のセキュリティ原則を適用する

学習目的・概要

- **目的**：CPTED（Crime Prevention Through Environmental Design：防犯環境設計）の考え方を、情報システムの設計・運用へ応用する。
- **物理セキュリティの役割**：①人命・身体の安全、②情報・設備の盗難や破壊の防止、③情報処理業務の継続を確保する。
- 意図的脅威（侵入・窃盗・破壊行為）と偶発的脅威（火災・停電・事故・自然災害）の両方を考える。
- **CISSPの優先順位**：人命の保護が最優先。情報システムが稼働していても、利用者が安全に使えなければ十分ではない。

防犯環境設計（CPTED）

基本思想

CPTEDは、建築・景観・人の動線・管理運用を組み合わせ、犯罪が起こりにくい環境を設計する考え方。**施設を単に要塞化するのではなく、日常的な使いやすさと安全性を両立**する。

- **割れ窓理論（Broken Windows Theory）**：放置された破損や荒廃が、秩序の欠如を示すという考え方。CPTEDと関連づけて説明されるが、理論の実証的妥当性や効果の大きさには議論がある。
- 整理整頓、修繕、照明、見通しの確保などは、犯罪機会の低減に役立つ。

CPTEDの原則と具体策

原則	目的	読み上げ内容に対応する例
自然監視（Natural Surveillance）	不審な行為が見えやすい環境をつくる	出入口・通路の照明、死角の削減、植栽の高さ・配置を工夫
自然アクセス制御（Natural Access Control）	人の動線を誘導し、無断侵入を難しくする	植栽や低木で進入経路を制限、出入口を集約、施錠
領域性の強化（Territorial Reinforcement）	公共・従業員・制限区域の境界を明確にする	重要区域を一般の動線から分離、区域表示、境界設計
維持管理（Maintenance）	荒廃・設備不良を放置しない	照明の修理、景観の整備、施錠設備の点検

原則	目的	読み上げ内容に対応する例
活動支援（Activity Support）	正当な利用者の活動を促し、自然な監視を生む	利用目的に応じた人の配置・動線設計

施設の物理的防護

対策・用語	内容・狙い	CISSP上の注意点
重要区域の隔離	サーバールーム等を一般来訪者・通常従業員の動線から分離	最小権限・職務上の必要性に基づく入退室制御
照明	出入口・駐車場・通路を見やすくし、侵入を抑止	明るさだけでなく死角・逆光・停電時照明も考慮
植栽・低木	景観を維持しつつ、進入経路の誘導・遮断に利用	茂みが隠れ場所にならないよう設計
ソリッドコアドア（Solid-core Door）	中空ドアより破壊・こじ開けへの抵抗性が高い	ドア枠・蝶番・錠前も一体で評価
ターンスタイル（Turnstile）	一人ずつ通過させ、通行を管理する回転ゲート	共連れ（Tailgating）対策に有用。ただし単独では完全に防げない
マントラップ（Mantrap）	二重扉などで通過者を一時的に区画し、認証・入退室を制御	非常時の避難経路・消防法規との両立が必要
鍵・錠前	最も基本的な物理的アクセス制御	鍵の発行・紛失・返却・交換を管理
窓の配置	扉・施錠機構の近くに窓を設けると、ガラス破壊による解錠の危険がある	窓から手を伸ばして開錠できない設計
合わせガラス（Laminated Glass）	中間膜で破片の飛散を抑え、貫通しにくくする	強化ガラス（Tempered Glass）とは性質が異なる
格子・メッシュ・窓ガード	窓からの不法侵入を抑止	非常脱出・消防規定への配慮
警備員による解錠	人が解錠を判断する方式	脅迫・なりすまし・ソーシャルエンジニアリングに備える。権限分離、手順、訓練が必要

ポイント：CPTEDの自然な環境設計と、ドア・ゲート・警備員などの物理的／運用的コントロールを**多層防御**として組み合わせる。

サイト計画（Site Planning）

- 主要拠点の立地は、顧客・交通・人材・費用などビジネス上の要件によって決まることが多い。
- **代替サイト（Alternate Site）**は、主要サイトと同一の災害・停電・通信断・洪水等で同時被災しないよう、**地理的・インフラ的な独立性**を検討する。
- 距離だけでなく、共通の電力系統、通信事業者、河川流域、地震帯、交通アクセスなども評価する。

- 代替処理能力は、別の物理サイト、クラウドサービスプロバイダー（CSP）、社内・社外の代替処理サービスへのフェイルオーバー等で確保できる。
- 選定後は、**維持管理・起動・切替・テスト・復旧の責任者**を明確化し、実際に利用可能な状態を維持する。

補足：代替サイトの種類

種類	状態	復旧の傾向
ホットサイト（Hot Site）	機器・環境・データが整備され、迅速に切替可能	速い／高コスト
ウォームサイト（Warm Site）	一部設備・データが用意され、追加作業が必要	中程度
コールドサイト（Cold Site）	場所・基本設備が中心で、機器設置やデータ復旧が必要	遅い／低コスト

※ ホット／ウォーム／コールドの分類は、この読み上げ部分への**試験対策補足**。復旧時間は設計・契約次第であり、分類名だけでは保証されない。

可用性（Availability）

- **狭義**：情報システムの稼働時間、サービスのアップタイム。
- **広義**：組織の目的達成に必要な人員、建物、電力、通信、設備、プロセス、アクセス経路まで含む。
- 例：サーバーが正常稼働していても、施設の閉鎖、停電、ネットワーク断、認証不能などで利用者が必要な業務を行えなければ、可用性は損なわれる。
- **CISSPの判断軸**：「サーバーが動いているか」ではなく、**必要な利用者が、必要なとき・場所・方法でサービスを利用できるか**。

建築基準法・建築設計

- 建物の安全確保は古くから制度化されており、現在は建築・消防・電気・設備などの規則と密接に関わる。
- 建物を**Systems of Systems（SoS：複数のシステムが相互依存する体系）**として捉える。構造、電力、空調、通信、消火、アクセス制御などは互いに影響する。
- 適切な建築設計は、**安全性・信頼性・レジリエンス（回復力）・運用コスト**に影響する。
- **IBC（International Building Code：国際建築基準）**は、建築安全に関するモデルコード。採用・適用される内容は管轄地域によって異なる。日本でIBCがそのまま法令になるわけではない。

データセンター標準

主要4規格の比較

規格・体系	策定主体	主な対象・特徴	覚えるポイント
Uptime Institute Tier Classification	Uptime Institute	データセンターの設備インフラの耐障害性・保守性をTier I〜IVに分類	**Tier III＝同時保守可能、Tier IV＝耐障害性**
ANSI/BICSI 002-2014	BICSI	データセンターの設計・構築・実装に関するベストプラクティス	**設計・実装の実務指針**
ANSI/TIA-942-A-2014	TIA	データセンターの通信インフラを中心に、関連する施設要件を規定	**通信・配線インフラ**

規格・体系	策定主体	主な対象・特徴	覚えるポイント
EN 50600	欧州の標準化体系	データセンター施設・インフラの設計・運用に関する規格群	**建築・電力・空調・配線・セキュリティ・運用を横断**

※ 表中の **2014** は教材で取り上げた版。規格は改訂されるため、実務で適用する際は最新版・契約上の指定版を確認する。BICSIの正式表記は **BICSI**（「BISCI」ではない）。

Uptime InstituteのTier分類

Tier	正式な特徴	設計上の意味	覚え方
Tier I	Basic Site Infrastructure	基本設備。計画保守等でサービス停止が必要になり得る	**基本**
Tier II	Redundant Site Infrastructure Capacity Components	UPS・冷却設備等の**容量コンポーネント**を冗長化。ただし配電経路等に単一経路が残り得る	**設備の冗長化**
Tier III	Concurrently Maintainable Site Infrastructure	設備の計画保守を、IT負荷を停止せず実施できるよう設計	**止めずに保守**
Tier IV	Fault Tolerant Site Infrastructure	単一の設備故障・事象に対してIT負荷を継続できるよう設計	**単一故障に耐える**

混同注意：Tier IIの「冗長化」≠ Tier IIIの「同時保守可能」≠ Tier IVの「フォールトトレラント」。Tier分類は施設インフラの設計評価であり、すべての障害や災害に対する無停止を保証するものではない。

EN 50600の構成（教材で登場した範囲）

パート	英語名	対象・概要
EN 50600-1	General Concepts	全体概念、分類、基本要求
EN 50600-2-1	Building Construction	建物構造・立地・建築要件
EN 50600-2-2	Power Distribution	受配電、電力供給・冗長性
EN 50600-2-3	Environmental Control	空調、温湿度などの環境制御
EN 50600-2-4	Telecommunications Cabling Infrastructure	通信ケーブル・配線インフラ
EN 50600-2-5	Security Systems	物理的セキュリティ設備・入退室管理など
EN 50600-2-6	Management and Operational Information Systems	施設管理・運用情報システム

CISSP試験直前チートシート

問題文のキーワード	判断する概念	最重要ポイント
景観・植栽・照明・犯罪抑止	**CPTED**	環境設計で犯罪機会を減らす
放置された破損・荒廃	**割れ窓理論**	秩序の維持と犯罪抑止の関連概念
見通し・死角・照明	**自然監視**	不審行為を見えやすくする
動線誘導・進入経路の制限	**自然アクセス制御**	無断侵入を難しくする
制限区域・公共区域の境界	**領域性の強化**	区域の所属と管理責任を明確にする
回転ゲート	**ターンスタイル**	通行を一人ずつ管理
二重扉で認証	**マントラップ**	共連れ対策・高セキュリティ区域
窓からの侵入防止	**合わせガラス／格子**	侵入抵抗と避難安全を両立
本番拠点と同時被災しない	**代替サイト計画**	地理的・インフラ的な独立性
サーバー稼働中なのに業務不可	**可用性の広義の捉え方**	利用者の実際の業務遂行を基準にする
建築・消防・設備の安全要件	**IBC等の建築基準**	現地法令との整合が必要
データセンターの基本設備	**Tier I**	基本
UPSなどの設備冗長化	**Tier II**	容量コンポーネントの冗長化
計画保守中もIT負荷を継続	**Tier III**	同時保守可能
単一設備故障でも継続	**Tier IV**	フォールトトレラント
データセンター設計・実装の指針	**BICSI 002**	ベストプラクティス
通信配線・通信インフラ	**TIA-942**	テレコミュニケーションインフラ
欧州の包括的施設規格	**EN 50600**	建築・電力・空調・通信・防護・運用

まとめ：CISSPの判断順序

1. **人命・身体の安全**を最優先する。
2. CPTEDによって、犯罪を起こしにくい環境を設計する。
3. 重要設備を隔離し、入退室管理・施錠・監視を**多層化**する。
4. 代替サイトの独立性と、運用・保守・切替の**責任分担**を確保する。
5. 可用性は設備単体ではなく、**業務を実行できる状態**として評価する。
6. Tier分類と各データセンター規格の**評価対象の違い**を押さえる。

3.9 サイトおよび施設のセキュリティコントロールを設計する

目的・概要

- **目的**：施設・システム・インフラストラクチャが情報セキュリティ（CIA：機密性・完全性・可用性）に与える影響を特定し、適切な物理的・環境的コントロールを設計する。
- **対象**：配線クローゼット、サーバールーム、ストレージメディア、証拠保管室、制限区域、HVAC、災害・火災・電力など。
- **判断基準**：リスクアセスメントを行い、予防・検知・対応・復旧を組み合わせる。**人命の安全を最優先**する。

1. 配線クローゼット・中間配線設備（IDF）

通信配線インフラ（**ケーブルプラント**）はシステムの信頼性・セキュリティの基盤。IDF（Intermediate Distribution Frame）はフロア等の中間配線拠点。MDF（Main Distribution Frame）は主配線拠点。

構成要素	役割・注意点
引込設備（Entrance Facility）	通信事業者・ISP回線の建物への引込点
通信事業者設備・特別回線	外部通信との接続、責任分界点の管理
設備室・通信室	ルータ、スイッチ、パッチパネル等を収容
バックボーン配線	MDF・IDF・通信室間を接続する幹線
水平配線	IDF等から各端末・情報コンセントへ接続

コントロール	対策例
物理アクセス	施錠、入退室認証、アクセス記録・監視、ラックの追加施錠
配線保護	導管、ケーブルトレイ、端子・パッチパネルの保護、盗聴・切断対策
環境	温湿度管理、換気・空調、漏水対策
電力	サージ・雷保護、UPS、バックアップ電源
火災・安全	火災検知・消火設備、緊急電源遮断（EPO）と誤操作防止

試験ポイント：通信設備はネットワーク機器だけでなく、ケーブル経路・引込点・配線室を含めて守る。

2. サーバールーム・データセンター

- 機器の最大負荷と必要稼働時間に応じた**UPS容量・バックアップ電源**を確保する。
- 電力サージ、停電、過熱、漏水、火災、無断入室への対策を多層化する。
- 高電圧・大電力設備では感電やアーク等の**人的安全**を重視する。
- **EPO（Emergency Power Off）**は緊急時に電源を遮断する装置。人命保護に役立つ一方、誤作動は可用性を損なうため、配置・表示・アクセス管理が重要。
- 火災検知・消火は電子機器への影響と人の避難を両方考慮する。

3. ストレージメディアの保護と管理

クラウド利用時でも、物理メディア・バックアップ・記録媒体の保護責任がなくなるわけではない。

ライフサイクル	管理策
作成・取得	媒体の分類、識別・台帳登録、暗号化
利用・移送	最小権限、持出し承認、追跡可能な配送、改ざん防止
保管	施錠、入退室管理、温湿度・防火・防水対策、必要ならオフサイト保管
保持	法令・契約・業務要件に応じた保存期間、バックアップの定期検証
廃棄・再利用	データ消去、暗号学的消去、媒体破砕等を機密度に応じ選択し、記録を残す

- 媒体ごとに許容温度・湿度・磁気・光などの条件が異なる。
- **CISSP視点**：データの所在がクラウドに変わっても、データ所有者のガバナンス・契約上の責任は残る。

4. 証拠保全と管理の連鎖（Chain of Custody）

証拠が**いつ、誰によって、どこで、何の目的で、どのように扱われたか**を追跡可能にする仕組み。法的証拠能力や社内調査の信頼性を支える。

段階	要点
収集	収集者、日時、場所、取得方法、対象を記録
識別・封印	一意の証拠ID、封印、ラベル、必要に応じハッシュ値
移送・受渡し	引渡者・受領者・時刻・理由・保管先を連続記録
保管	アクセス制限、施錠・監視、環境保護、閲覧記録
分析	原本保護、検証可能な複製で分析、手順・結果を記録
提出・廃棄	権限・保持要件に従い処理し、最終処分も記録

重要：ハッシュ値は証拠のビット単位の同一性を示す補助手段。**Chain of Custodyは人的な受渡し・保管履歴を含む**ため、ハッシュだけでは代替できない。記録に欠落があると証拠の信用性や採用可能性に影響する。

5. 作業制限区域のセキュリティ・SCIF

SCIF（Sensitive Compartmented Information Facility）：高度に機密性の高い情報を取り扱うため、物理・音響・視覚・電子的な漏えい経路を考慮して設計・認定される制限区域。

防護対象	例
壁・床・天井・扉	侵入耐性、堅牢なドア、開口部・貫通部の保護
音響	会話・振動等の外部への漏えい抑止（遮音）
視覚	窓・視線・監視可能な周辺空間への対策、覗き見防止
電気・通信	承認済み配線・設備、不要な接続の排除、信号漏えい対策
ダクト・配管	侵入防止用の障壁、点検可能性、貫通部の保護
侵入検知	壁・床・天井・出入口等へのセンサー、警報対応
運用	身元確認、権限に基づく入室、訪問者管理、定期点検

※ 読み上げの「音響保護は外の人が音を検知できる」は逆で、**外部から機密会話を聞き取れないようにする**のが目的。

6. ユーティリティ・HVAC

- **HVAC（Heating, Ventilation and Air Conditioning）**：暖房・換気・空調。高密度IT機器の排熱、適切な温湿度・空気品質を維持する。
- 停電・冷却停止は可用性を直接損なうため、監視・アラート・冗長化・保守を設計する。

- ビル管理システム（BMS）や空調制御もネットワーク接続される場合があり、アクセス制御・セグメンテーション・変更管理が必要。
- 電源、水道、冷却、通信等のユーティリティは単一障害点と依存関係を確認する。

7. 環境上の脅威

脅威	主なリスク・対策
ハリケーン・竜巻・暴風	建物損傷、停電、通信断：立地・耐風設計・代替サイト
森林火災・山火事	延焼、煙、電力停止：防火帯・空調保護・避難計画
地震	ラック転倒、配管破損：耐震固定・構造対策
津波・洪水	浸水、感電、設備喪失：浸水想定確認、重要機器の高所配置
土砂崩れ	建物・道路損壊：地盤調査・代替輸送経路

要点：災害リスクは立地・施設設計・事業継続計画（BCP）を横断して扱う。

8. 防火・火災検知・消火

火災対策は**予防 → 早期検知 → 警報・避難 → 消火 → 復旧**の順で考える。従業員の教育・訓練と避難経路の確保が不可欠。

火災クラス（米国で一般的な分類）

クラス	対象	例・注意
A	普通可燃物	紙、木材、布、一部のプラスチック
B	可燃性液体・ガス等	ガソリン、溶剤など
C	通電中の電気設備	通電状態が危険要因。電源遮断後は燃焼物のクラスで扱う
D	可燃性金属	マグネシウム、ナトリウム等。専用消火剤
K	調理用油脂	業務用フライヤーなど。専用湿式薬剤

※ 国・規格によって分類が異なる。例えば欧州・日本の分類と米国の**Class C/K**を機械的に同一視しない。

水系消火設備の比較

方式	配管の状態・作動	特徴・適用例
Wet Pipe（湿式）	常時水が充填、ヘッド作動で放水	構造が単純、応答が速い。凍結に弱い
Dry Pipe（乾式）	通常は加圧空気等、作動後に水が流入	凍結しやすい場所向け。放水に遅延あり
Preaction（予作動式）	通常は配管に水を満たさず、検知設備との連動等で給水	誤放水を抑えたいデータセンター等で利用。方式により作動条件が異なる

方式	配管の状態・作動	特徴・適用例
Deluge（一斉開放式）	開放型ヘッド、作動時に区域全体へ一斉放水	急速な延焼が懸念される高危険区域向け

- 水は主として**Class A**に有効。通電設備や油火災などには安易に使用しない。
- **水性泡消火剤**は可燃性液体等の火災で使われる場合がある。薬剤は燃料の種類・設備条件・環境規制に適合させる。

ガス系・クリーンエージェント

薬剤・方式	特徴・注意点
HFC（ハイドロフルオロカーボン）	ハロカーボン系。電気設備に残留物を残しにくい製品があるが温暖化影響・規制を考慮
FM-200（HFC-227ea）	ハロカーボン系の代表例。HFCの一種
ハロン	歴史的に使用。オゾン層破壊のため新規利用は厳しく制限
CO_2（二酸化炭素）	酸素濃度低下による消火。**人命への窒息危険が大きく、有人空間では特に注意**
Argonite（アルゴナイト）	アルゴンと窒素の混合不活性ガス。酸素濃度を下げて消火
Inergen（イナージェン）	窒素・アルゴン・CO_2の混合不活性ガス。酸素濃度低下に注意

- 適切に設計されたガス系設備は、薬剤・用途に応じ**Class A/B/C**相当の火災に対応し得る。すべてのガスがあらゆる火災に適するわけではない。
- 人の在室可能性、避難警報、放出遅延、室内密閉性、設備・環境への影響を評価する。
- **CISSP最重要**：機器保護より人命保護を優先する。

9. 電力・電源品質

発電所 → 高圧送電 → 変電所 → 配電 → 施設内の受配電設備 → IT機器、という供給経路。データセンターでは電源品質と冷却設備の両方が可用性に直結する。

用語	意味・対策
Blackout	停電。UPS・発電機・代替サイトで備える
Brownout	電圧低下。UPSや電源調整装置で対処
Sag / Dip	短時間の電圧低下
Surge / Spike	過電圧・瞬間的電圧上昇。サージ保護装置（SPD）等
Noise	電気的ノイズ。フィルタ・適切な配線・接地等
UPS	短時間の停電を橋渡しし、電源品質を安定化する。必要容量・保持時間の設計が重要
発電機	長時間停電への備え。起動時間、燃料、保守、定期試験を考慮
EPO	緊急時の電源遮断。人命安全と誤操作対策の両立が必要

- **UPSと発電機は役割が異なる**：UPSは即時給電・橋渡し、発電機は長時間給電。
- データセンターの電力消費割合は年・統計範囲で変動するため、教材の「世界の電力の1〜2%」は固定値として暗記しない。
- 「発電・送電で60%以上損失」は一般的な送配電損失率を示す説明としては不正確。発電方式の熱効率と送配電損失を区別する。

CISSP直前チートシート

問題文のキーワード	思い出すべき論点
通信室、配線、ISP引込	MDF/IDF、ケーブルプラント、物理アクセスと環境保護
データセンターの短時間停電	UPS
長時間停電	発電機、燃料、BCP
感電・緊急停止	EPO、人命最優先
クラウド利用と媒体管理	データ所有者の責任、保持・安全な廃棄
証拠の受渡し履歴	Chain of Custody
証拠データの改ざん確認	ハッシュ値＋保管・手続記録
機密区画・盗聴防止	SCIF、音響・視覚・電磁・物理的対策
温度・湿度・排熱	HVAC、監視・冗長化
水損害を避けたいサーバールーム	予作動式スプリンクラー（Preaction）の検討
凍結のおそれがある配管	乾式（Dry Pipe）
一斉放水が必要な高危険区域	Deluge
普通可燃物	Class A
可燃性液体	Class B
通電設備	Class C（米国分類）
金属火災	Class D
調理油脂	Class K（米国分類）
ガス消火と人の安全	CO_2の窒息危険、避難設計
消火剤の環境影響	ハロンのオゾン層破壊、HFCの温暖化影響

重要な区別・教材補足

1. **3.8はサイト選定・CPTED・施設設計の原則、3.9は配線・設備・証拠・空調・火災・電力など具体的なコントロール設計。**
2. **SCIFは機密情報の保護区画**であり、単に施錠したサーバールームとは要求が異なる。
3. **証拠保全（Chain of Custody）は技術だけでなく記録と責任の連続性が重要。**
4. **火災クラス・消火剤の適合性は地域の消防規格に依存**する。教材の米国式分類を基準に整理した。
5. **人命安全 → 事業継続 → 資産保護**の優先順位を意識する。

3.10 情報システムライフサイクルを管理する

目的と概要

- **目的**：情報システムのライフサイクル全体を管理するためのプロセスとタスクを理解する。
- **基本思想**：システムを導入して終わりではない。ニーズの把握から要件定義、設計、開発、統合、検証、配備、運用、廃止、データ処分まで、セキュリティを一貫して管理する。
- メインフレームから分散処理、クラウドへの移行のような大規模変化だけでなく、プロセス・構成・技術の小さな変更も管理対象。
- **Security by Design / Shift Left**：セキュリティを最後に付け足さず、早期フェーズから組み込む。

ライフサイクルの全体像

フェーズ	主な活動	セキュリティ上の焦点
利害関係者のニーズと要件	ステークホルダー特定、要求収集、優先順位付け	法令・プライバシー・データ分類・追跡可能性
要件分析	要件の精査、ユースケース・ミスユースケース	悪用シナリオとセキュリティ要件の具体化
アーキテクチャ設計	代替案比較、脅威モデリング	多層防御、攻撃面・データの最小化
開発・実装	セキュアコーディング、ライブラリ管理	IDE支援、静的解析、依存関係管理
インテグレーション	上流・下流・レガシーシステムとの結合	インターフェース・認証・データフローの検証
検証・妥当性確認	要件別テスト、SAST/DAST、ASVS	実装の正しさと目的への適合
移行・配備	CI/CD、リリース管理	承認、変更管理、パイプライン保護
運用・保守・維持	ITSM、ISCM、脆弱性対応、監視	検知・対応・継続的なリスク管理
自動化	定型検査・監視・配備の自動化	効率、再現性、監視の信頼性
廃止・廃棄	システム停止、資格情報失効、記録保管	残存資産・データ・権限の排除
データアーカイブ・廃棄	長期保存、保持期限後の処分	保持義務、機密性、復元不能化

1. 利害関係者のニーズと要件

- **利害関係者マップ（Stakeholder Map）**：利用者、事業部門、運用担当、セキュリティ、法務、規制当局などの関係と期待を可視化。
- **要件トレーサビリティマトリックス（RTM）**：各要件を設計・実装・テストケース・検証結果に紐付け、未実装や未検証を防ぐ。
- **機能要件（Functional Requirements）**：「何をするか」。業務プロセス、ユースケース、業務ルール、提供機能。
- **非機能要件（Non-functional Requirements）**：「どの程度・どの条件で動くか」。性能、可用性、拡張性、監査可能性、プライバシーなど。セキュリティ要件には機能として記述されるものもあるため、すべてが非機能要件とは限らない。

- 法令、プライバシー規制、業界標準、データ分類、品質、制約を考慮する。
- 要件管理の流れ：**収集 → 定義・明確化 → 優先順位付け → 文書化 → 実装への追跡 → 検証**。

2. 要件分析

- ユースケース（正規の利用）に加え、**ミスユースケース／アビュースケース**（攻撃者による悪用）を検討。
- 処理データの分類を確認し、適用法令・コンプライアンス要件を導出する。
- 業務フロー・データフローを明らかにして、必要なセキュリティ制御を要件化する。
- **Microsoft SDL（Security Development Lifecycle）**：開発の各段階にセキュリティ活動を組み込む代表的な方法論。

3. アーキテクチャ設計

- 複数の設計案を、要件への適合性・リスク・運用可能性などで比較する。
- **脅威モデリング**：攻撃者、資産、信頼境界、攻撃経路を分析し、対処すべき脅威を列挙する。
- **自己防衛できる設計**：外周防御だけに依存せず、各コンポーネントが認証・認可・入力検証・ログなどを備える。
- **セキュアコーディング、攻撃面（Attack Surface）の削減、データ最小化、多層防御（Defense in Depth）**を採用する。

セキュリティアーキテクチャの特性・原則

原則・要素	意味	具体例／試験での注意
オープンデザイン（Open Design）	安全性を設計の秘密に依存させない	アルゴリズムを公開しても、鍵などの秘密が守られれば安全である設計。Kerckhoffsの原則に通じる
完全な仲介（Complete Mediation）	保護対象への**すべてのアクセス**を認可検査する	初回だけ確認して以降の権限変更を見逃さない。キャッシュによる検査省略に注意
マイクロセグメンテーション	ネットワークやワークロードを細分化し、通信を制限	侵害後の横展開を抑止
ゼロトラスト	場所やネットワーク内外だけで暗黙に信頼しない	ID、端末状態、コンテキストなどに基づき継続的に評価
共通メカニズムの最小化（Least Common Mechanism）	異なる主体が共有する仕組み・経路をできるだけ減らす	共有DB接続や共有プロキシが意図しない情報交換の経路にならないよう分離
区画化・分離（Compartmentalization）	情報・機能・権限を区切り、侵害範囲を限定	プロセス分離、テナント分離、権限境界。読み上げの「クカクカ」はこの概念の可能性があるため仮置き
ライブラリの安全な再利用	検証済み・保守中のコンポーネントを利用	自作暗号や独自認証を避け、依存ライブラリの更新・脆弱性管理も行う

共通メカニズムの最小化の補足：複数アプリケーションが同一の高権限DB接続を共有すると、1つのアプリケーションの侵害が別アプリケーションのデータに波及し得る。アクセス経路・資格情報・権限を適切に分離する。これは**最

小権限とは別の原則だが、併用すると効果的。

4. 開発・実装

- セキュアな開発環境、検証済みの最新ライブラリ、保護されたソースコードリポジトリを整備する。
- **IDE**：統合開発環境。**Linting**：コード規約違反や疑わしい書き方を早期検出。
- コード登録・プルリクエスト時に静的解析を自動実行する。
- **用語注意**：静的コード解析は一般に **SAST（Static Application Security Testing）**。**SCA** は通常 **Software Composition Analysis**（OSSなどの依存コンポーネント解析）を意味する。教材の「静的コード分析、SCA」は混同の可能性があるため、試験では区別する。
- 依存ライブラリの脆弱性・ライセンス・バージョンを継続管理する。

5. インテグレーション（統合）

- 現実のシステムは、上流・下流のサービス、外部API、既存システムと相互依存する。
- **結合テスト（Integration Testing）**：コンポーネント、モジュール、システム間のインターフェースとデータ受け渡しを検証。
- **レガシーシステム**との連携では、古い認証方式、暗号、通信仕様、パッチ制約などを評価する。
- 接続境界ごとの認証・認可、入力検証、暗号化、エラー処理、ログを確認する。

6. 検証と妥当性確認

- **検証（Verification）**：「仕様どおり正しく作ったか」。設計・実装・テスト結果が要件を満たすか。
- **妥当性確認（Validation）**：「利用者・事業の目的に合ったものを作ったか」。実際のニーズに適合するか。
- **RTM** に各要件のテストケースと結果を紐付け、抜け漏れを把握する。
- **OWASP ASVS（Application Security Verification Standard）**：アプリケーションのセキュリティ要件・検証項目を体系化した標準。リリースに応じて該当項目を選び、テスト可能な要件やユーザーストーリーへ落とし込む。
- **TTPs（Tactics, Techniques, and Procedures）**：攻撃者の戦術・技術・手順。脅威モデリングで特定したTTPへの対策が設計・実装で機能するか確認する。

手法	何を調べるか	実行の有無	主な特徴
SAST	ソースコードやバイナリの脆弱性	原則、対象アプリの実行不要	開発早期に検出しやすい。誤検知や実行時の文脈不足があり得る
DAST	稼働中アプリの外部から観測できる問題	実行が必要	実際の入出力・挙動を検査。内部コードの網羅性には限界
SCA	OSS・サードパーティ依存関係	ソース・依存情報等を解析	既知脆弱性、ライセンス、部品構成を確認
結合テスト	コンポーネント間の連携	実行が必要	API、認証連携、データ整合性などを検証

7. 移行・配備：CI/CD

- **CI（Continuous Integration：継続的インテグレーション）**：変更を頻繁に統合し、自動ビルド・テストなどで問題を早期発見。

- **CD** には異なる2つの意味がある。**継続的デリバリー（Continuous Delivery）と継続的デプロイメント（Continuous Deployment）**を区別する。

項目	継続的デリバリー	継続的デプロイメント
CIの活用	あり	あり
本番に出せる状態の維持	する	する
本番リリース	**人の承認・手動判断を経て実施**	**パイプライン通過後に自動で実施**
キーワード	「いつでもリリース可能」	「合格した変更を自動リリース」

- 配備パイプラインでは、テスト、承認、変更記録、シークレット保護、アーティファクトの完全性、ロールバック手段を確保する。
- **重要**：自動配備でも、セキュリティ審査やガードレールが不要になるわけではない。

8. 運用・保守・維持

- 本番環境では可用性・性能・健全性に加え、**不正利用・侵入・異常・脆弱性**を監視する。
- **ITSM（IT Service Management）**：インシデント、問題、変更、構成、サービス提供などを管理する枠組み。
- **CMDB（Configuration Management Database）**：構成アイテムとその関係を管理。脆弱性・ゼロデイ発見時に影響システムを特定するために役立つ。
- コードリポジトリ、依存関係情報、資産台帳と突き合わせ、必要なパッチ・アップグレードを特定・展開する。
- 本番の新機能・変更は、定められた変更・リリース管理を経てセキュリティテストする。
- **ISCM（Information Security Continuous Monitoring：情報セキュリティ継続的監視）**：継続的にセキュリティ状況・コントロールの有効性・リスク変化を把握し、意思決定につなげる。

運用監視の目的

- 攻撃の**早期発見**、より迅速で適切な対応、被害の軽減。
- 社内ポリシー・標準違反の検知。
- **HIPAA、SOX、GDPR** など適用される規制・法令への対応（法令ごとの適用範囲に注意）。
- **デジタルフォレンジック**のための証跡保全、インシデント調査。
- 継続的な評価や必要に応じた**再認可（Reauthorization）**の支援。

セキュリティ監視・攻撃検知の実装

要素	目的・留意点
イベント収集・集約	大量のログ・イベントを集中管理し、横断分析可能にする
中央リポジトリ	保持期間、アクセス制御、改ざん防止、時刻同期を考慮
相関分析・検知ルール	シグネチャ、異常パターン、複数イベントの関連を検出
対応体制	アラートのトリアージ、エスカレーション、インシデント対応を定義
監視コントロールの保護	監視の停止・設定変更・ログ削除の権限を制限し、操作を監査

9. 自動化

- 自動化は**コスト削減、効率向上、処理の一貫性、監視の信頼性向上**に有効。
- 例：CI/CD、SAST・SCA、構成チェック、パッチ適用、ログ相関、定型インシデント対応。
- 自動化の誤設定は大規模な影響を招くため、権限分離、承認、テスト、監査ログ、停止・復旧手段が必要。

10. 廃止および廃棄

システムやソフトウェアの廃止は、単なるアンインストールではなく、**残存するデータ・権限・設定・契約・証跡の管理**を含む。

作業	内容
ソフトウェア削除	実行可能コード、ヘルプファイル、関連プログラムを安全にアンインストール
関連ファイル削除	サポートファイル、データファイル、文書、ディレクトリ構造の所在を把握し処理
構成・接続の撤去	設定、ジョブ、連携、ネットワーク経路、監視設定などを整理
資格情報の失効	パスワード、APIキー、証明書、サービスアカウントなどを無効化・削除
ライセンス・契約	利用権、サブスクリプション、保守契約の解約・精算
アーカイブ	必要なソースコード・文書・記録を安全に保存
廃止証跡	実施日時、担当者、タイムスタンプ、必要に応じた電子署名、処理記録を保管

11. データアーカイブ

- **アーカイブ（Archiving）**：本番システム上の非アクティブデータを特定し、長期保存向けのストレージへ移すこと。
- 保存期間、検索・復元の必要性、暗号化、アクセス制御、法的保全（Legal Hold）を考慮する。
- **バックアップとの違い**：バックアップは主に障害・誤操作からの復旧用。アーカイブは長期保存・記録管理が中心。

12. データ廃棄とメディアサニタイズ

- ソフトウェア廃止時も、法令・契約・訴訟保全などの**データ保持要件**を先に確認する。保持義務があるデータは即時消去しない。
- **メディアサニタイズ（Media Sanitization）**：媒体に残る機密情報を、リスクに応じた方法で利用不能にすること。
- 情報は、紙のリサイクル箱、修理・返品機器、交換されたホットスワップ部品、廃棄ディスクなどから組織外に流出し得る。

方法	概要	注意点
Clear（クリア）	通常の手段でデータを読み取れないようにする論理的処理	高度な復元への耐性は方式に依存
Purge（パージ）	より高度な復元にも耐える処理（例：適切な暗号学的消去）	媒体と暗号鍵の管理状況を確認

方法	概要	注意点
Destroy（破壊）	物理的に媒体を破壊	再利用不可。破壊粒度や処理証明が重要

- 廃棄時は媒体の分類、責任者、保管・搬送の記録、処理業者の管理、処理証明を確認する。

CISSP試験直前チートシート

キーワード	即答ポイント
RTM	要件 → 実装 → テスト → 結果を追跡
機能要件 / 非機能要件	何をするか / 品質・性能・制約など
ミスユースケース	攻撃者による悪用を要件分析に取り込む
Microsoft SDL	開発工程にセキュリティ活動を統合
脅威モデリング	想定攻撃と信頼境界を分析
Open Design	秘密の設計に依存しない
Complete Mediation	**毎回**アクセス権を確認
Least Common Mechanism	共有機構・共有経路を最小化
マイクロセグメンテーション	横展開を抑える細粒度の分離
ゼロトラスト	暗黙の信頼を置かず継続的に評価
Lint / SAST / SCA	コード品質検査 / 静的脆弱性検査 / 依存部品解析
DAST	動作中のアプリを外部から検査
ASVS	OWASPの検証可能なアプリセキュリティ要件集
Verification / Validation	正しく作ったか / 正しいものを作ったか
CI	継続的に統合し自動テスト
Continuous Delivery	本番リリースに人の判断が入る
Continuous Deployment	パイプライン合格後、本番へ自動反映
ITSM	サービス運用管理の枠組み
CMDB	構成アイテムと依存関係の管理
ISCM	セキュリティ状態とリスクの継続監視
廃止	ファイルだけでなく設定・権限・ライセンスも処理
アーカイブ	長期保存。バックアップ（復旧目的）とは別
サニタイズ	Clear / Purge / Destroy をリスクで選ぶ

補足・用語の確認

- 読み上げ中の「クカクカ」は文脈から**区画化・分離**として仮整理した。原文の見出しが別語であれば修正する。
- 「暴力的コーディング」は文脈上**防御的／セキュアコーディング**として解釈した。
- 「静的コード分析、SCA」は一般的な略称と異なるため、**SASTとSCAを分けて**記載した。
- 本資料は読み上げ内容を基礎に、試験対策に役立つ一般的な説明を補足したもの。法令・標準の版や具体的な適用条件は別途確認する。